

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO BÀI TẬP LỚN

CÁC HỆ THỐNG PHÂN TÁN

**Chủ đề 3: Phát triển hệ thống chat ngang hàng P2P
(Peer-to-Peer Chat System)**

Nhóm thực hiện: 15

THÀNH VIÊN THỰC HIỆN

1. Nguyễn Hồng Hải - B22DCCN268
2. Trần Trọng Thái - B22DCCN784

*Giảng viên hướng dẫn: **Kim Ngọc Bách***

Hà Nội – 05/2026

Chương 1: Giới thiệu chung

Link Github: <https://github.com/haibadguy/p2p-chat-project.git>

1.1. Đặt vấn đề

Trong bối cảnh mạng Internet phát triển mạnh mẽ, nhu cầu trao đổi thông tin theo thời gian thực đã trở thành một tiêu chuẩn thiết yếu đối với các nền tảng công nghệ. Hiện nay, hầu hết các ứng dụng nhắn tin phổ biến như Messenger, Zalo hay Telegram đều được xây dựng dựa trên kiến trúc client-server truyền thống. Trong mô hình này, toàn bộ luồng dữ liệu giao tiếp giữa các người dùng đều phải được luân chuyển qua một hệ thống máy chủ trung tâm. Mặc dù phương pháp tiếp cận này mang lại nhiều ưu điểm về khả năng quản lý, tính dễ triển khai và đảm bảo tính đồng bộ của thứ tự tin nhắn, song kiến trúc tập trung cũng đồng thời bộc lộ nhiều hạn chế mang tính hệ thống.

Cụ thể, việc phụ thuộc hoàn toàn vào máy chủ trung gian làm phát sinh các rủi ro và thách thức kỹ thuật đáng kể, bao gồm:

- Điểm lỗi đơn: Sự cố hoặc gián đoạn tại máy chủ trung tâm có thể dẫn đến tình trạng tê liệt toàn bộ hệ thống mạng lưới.
- Nút thắt băng thông: Khi lưu lượng truy cập và số lượng người dùng gia tăng đột biến, hệ thống dễ bị quá tải do mọi luồng dữ liệu đều phải tập trung xử lý tại một điểm duy nhất.
- Rủi ro về quyền riêng tư và bảo mật: Máy chủ trung tâm về mặt lý thuyết có khả năng thu thập, tiếp cận siêu dữ liệu hoặc nội dung tin nhắn nếu các cơ chế mã hóa không được áp dụng triệt để.
- Chi phí vận hành và bảo trì: Việc duy trì một hệ thống máy chủ quy mô lớn đòi hỏi nguồn lực tài chính khổng lồ cho hạ tầng phần cứng, băng thông và đội ngũ nhân sự kỹ thuật.

Để khắc phục những giới hạn trên, kiến trúc mạng ngang hàng (Peer-to-Peer - P2P) nổi lên như một giải pháp công nghệ thay thế tối ưu. Khác với mô hình tập trung, kiến trúc phân tán của P2P cho phép các nút mạng (peer) kết nối và giao tiếp trực tiếp với nhau mà không cần thông qua máy chủ trung gian để truyền tải dữ liệu. Đây là đặc trưng cốt lõi của các hệ thống phân tán, nơi quyền quản trị và năng lực xử lý được phân bổ đồng đều trên toàn mạng lưới thay vì tập trung hóa.

Xuất phát từ cơ sở lý luận và thực tiễn nêu trên, dự án ELITE P2P CHAT được đề xuất và tiến hành triển khai. Mục tiêu trọng tâm của dự án là nghiên cứu và xây dựng một hệ thống nhắn tin thời gian thực dựa trên kiến trúc phân tán toàn diện. Trong mạng lưới này, mỗi thiết bị của người dùng sẽ hoạt động song song với tư cách là một máy khách (client) và một máy chủ (server) độc lập. Các nút mạng có khả năng tự động lắng nghe kết nối đến, đồng thời chủ động thiết lập giao tiếp với các thực thể khác nhằm trao đổi tin nhắn, tệp tin và tham gia vào các nhóm trò chuyện một cách bảo mật, liền mạch và tối ưu tài nguyên.

1.2. Mục tiêu dự án

Mục tiêu tổng quát của dự án là thiết kế và xây dựng một hệ thống nhắn tin ngang hàng (P2P) hoạt động theo thời gian thực. Hệ thống phải có năng lực tự tổ chức mạng lưới, cho phép truyền tải tin nhắn trực tiếp giữa các nút mạng (peer) kết hợp cơ chế mã hóa đầu cuối (End-to-End Encryption - E2EE) nhằm đảm bảo tính bảo mật. Bên cạnh đó, hệ thống cần được thiết kế để xử lý hiệu quả các kịch bản đặc thù của môi trường phân tán như sự xáo trộn mạng khi người dùng liên tục tham gia hoặc rời đi (churn), tình trạng mất kết nối tạm thời và bài toán phân phối tin nhắn đến nhiều đích đến.

Để hiện thực hóa mục tiêu chung, dự án đặt ra các mục tiêu kỹ thuật cụ thể dựa trên kiến trúc phân tán theo mô hình có máy chủ hỗ trợ (Bootstrap-assisted P2P), phân tách rõ rệt giữa mặt phẳng điều khiển (quản lý thành viên) và mặt phẳng dữ liệu (truyền tải tin nhắn). Các mục tiêu trọng tâm bao gồm:

- Về giao thức và truyền tải: Triển khai giao thức truyền tin TCP/IP kết hợp định dạng JSON để đảm bảo giao tiếp ổn định. Xây dựng cơ chế đảm bảo độ tin cậy ở tầng ứng dụng bằng tín hiệu xác nhận (ACK) và chiến lược gửi lại tự động. Hệ thống phải hỗ trợ đa dạng các mô hình truyền tin như trò chuyện trực tiếp, phát sóng toàn mạng và trò chuyện nhóm.
- Về bảo mật: Tích hợp bảo mật E2EE thông qua thuật toán trao đổi khóa Diffie-Hellman và mã hóa dòng đối xứng, đảm bảo nội dung thông điệp miễn nhiễm hoàn toàn trước nguy cơ rò rỉ từ máy chủ Bootstrap.
- Về xử lý ngoại lệ và giao diện: Phát triển cơ chế lưu trữ và chuyển tiếp cho các thiết bị ngoại tuyến, phát hiện lỗi qua heartbeat định kỳ để loại bỏ các nút không phản hồi. Đồng thời, dự án cung cấp cả giao diện dòng lệnh (CLI) và giao diện đồ họa (GUI) để phục vụ công tác mô phỏng và kiểm thử.

1.3. Phạm vi dự án

1.3.1. Phạm vi chức năng

Dự án tập trung phát triển các tính năng cốt lõi phục vụ quá trình giao tiếp nội bộ trong mạng P2P. Bảng dưới đây tóm tắt các chức năng chính của hệ thống:

Tên chức năng	Mô tả chi tiết nghiệp vụ
Quản lý mạng	Cho phép Peer đăng ký/rời mạng với Bootstrap Server và gửi tín hiệu định kỳ.
Khám phá Peer	Lấy danh sách thành viên trực tuyến và nhận thông báo khi có sự thay đổi.
Chat 1-1 E2EE	Gửi/nhận tin nhắn mã hóa trực tiếp và bảo mật giữa hai điểm cuối.
Phát sóng (Broadcast)	Phát tin nhắn tới toàn mạng qua cơ chế epidemic flooding.
Trò chuyện nhóm	Tạo nhóm, thêm thành viên và gửi tin nhắn trong nhóm.
Chuyển tệp tin	Truyền tải tệp tin đã mã hóa E2EE (giới hạn dung lượng dưới 2MB).
Lưu trữ chờ (Store-and-forward)	Lưu tạm tin nhắn cho peer ngoại tuyến và tự động phân phối lại khi họ trực tuyến.
Mô phỏng mạng	Chạy các kịch bản kiểm thử việc peer gia nhập và rời mạng liên tục (churn).

1.3.2. Phạm vi kỹ thuật

Về mặt công nghệ, dự án sử dụng ngôn ngữ lập trình Python dựa trên các thư viện tiêu chuẩn, vận hành qua giao thức TCP/IP với kỹ thuật phân định khung dữ liệu bằng JSON. Hệ thống được định hướng triển khai và thử nghiệm hoàn toàn trong môi trường mạng cục bộ (LAN/localhost), chưa giải quyết bài toán vượt tường lửa cho mạng Internet công cộng. Dự án cũng chủ đích loại trừ các

yếu tố nằm ngoài trọng tâm nghiên cứu như cơ sở dữ liệu tập trung, cơ chế xác thực danh tính người dùng, gọi thoại/video hay bảng bấm phân tán.

1.4. Phương pháp thực hiện

Dự án được tiến hành một cách có hệ thống thông qua một quy trình chuẩn bao gồm năm giai đoạn tương hỗ lẫn nhau. Đầu tiên, bước nghiên cứu lý thuyết tập trung vào việc thu thập tài liệu về kiến trúc P2P, đặc tính của hệ thống phân tán và các tiêu chuẩn bảo mật. Tiếp đó, quá trình phân tích và thiết kế được thực hiện nhằm phác thảo kiến trúc các khối thành phần, chuẩn hóa định dạng giao thức và xây dựng sơ đồ luồng dữ liệu. Ở giai đoạn cài đặt, toàn bộ mã nguồn được phát triển bằng Python, ứng dụng triệt để mô hình đa luồng nhằm tối ưu hóa năng lực xử lý đồng thời. Sau khi hoàn thiện, dự án bước vào khâu kiểm thử nghiêm ngặt thông qua các tập lệnh unit test, kịch bản mô phỏng xáo trộn mạng và thử nghiệm nghiệp vụ trên hai nền tảng giao diện. Cuối cùng, bước đánh giá tiến hành đối chiếu kết quả đạt được với các mô hình truyền thống nhằm rút ra các ưu điểm, hạn chế và định hình hướng phát triển mở rộng cho hệ thống.

Chương 2: Cơ sở lý thuyết

2.1. Tổng quan về hệ thống phân tán

2.1.1. Định nghĩa và đặc trưng

Hệ thống phân tán là một tập hợp các thành phần tính toán độc lập, được kết nối với nhau thông qua mạng máy tính. Mặc dù cấu tạo từ nhiều phần cứng riêng biệt, hệ thống được thiết kế để phối hợp hoạt động nhằm thực hiện một mục tiêu chung, tạo cho người dùng cảm giác đang tương tác với một hệ thống máy tính thống nhất. Điểm khác biệt cốt lõi là hệ thống phân tán không chia sẻ bộ nhớ chung và không có đồng hồ vật lý đồng bộ hoàn hảo. Mỗi tiến trình hoặc nút mạng (node) sở hữu bộ nhớ cục bộ và trạng thái riêng biệt; quá trình giao tiếp và đồng bộ hóa diễn ra hoàn toàn thông qua cơ chế truyền thông điệp.

Các hệ thống phân tán được xác định thông qua năm đặc trưng cốt lõi, bao gồm:

- Tính đồng thời: Nhiều tiến trình hoặc nút mạng hoạt động và xử lý tác vụ song song cùng một thời điểm.
- Không chia sẻ bộ nhớ: Thiếu vắng bộ nhớ vật lý chung, buộc các nút phải trao đổi thông tin thông qua mạng.

- Khả năng chịu lỗi độc lập (Independent failure): Sự cố hoặc lỗi xảy ra tại một thành phần không dẫn đến sự sụp đổ của toàn bộ hệ thống; các phần khác vẫn có khả năng duy trì hoạt động.
- Tính không đồng nhất: Mạng lưới có thể bao gồm nhiều nút sử dụng kiến trúc phần cứng, hệ điều hành và tốc độ mạng khác nhau.
- Thiếu đồng hồ toàn cục: Do độ trễ mạng, hệ thống không có một điểm tham chiếu thời gian tuyệt đối để sắp xếp chính xác thứ tự các sự kiện trên toàn mạng.

2.1.2. So sánh kiến trúc tập trung và phân tán

Kiến trúc phân tán P2P mang lại sự thay đổi căn bản về mặt luồng dữ liệu và vai trò của các thực thể tham gia so với kiến trúc tập trung truyền thống (Client-Server). Sự khác biệt này được trình bày chi tiết trong Bảng 2.1.

Tiêu chí	Client-Server	P2P (Phân tán)
Vai trò các nút	Bất đối xứng (Client thụ động, Server xử lý trung tâm).	Đối xứng (Mọi nút đều có năng lực xử lý ngang hàng).
Luồng truyền tải	Mọi dữ liệu bắt buộc đi qua Server trung gian.	Trực tiếp giữa các nút (Peer-to-Peer) hoặc đa điểm nhảy (multi-hop).
Điểm lỗi đơn	Rủi ro cao tại Server trung tâm.	Tùy thuộc vào thiết kế (Rủi ro thấp trong P2P thuần túy).
Khả năng mở rộng	Bị giới hạn bởi năng lực của Server.	Tỷ lệ thuận với số lượng nút tham gia mạng.
Độ phức tạp kỹ thuật	Trung bình.	Cao (Do phải giải quyết bài toán khám phá nút, tính nhất quán và khả năng chịu lỗi).

2.2. Mô hình Peer-to-Peer (P2P)

2.2.1. Khái niệm P2P

Mô hình mạng ngang hàng (Peer-to-Peer - P2P) là một kiến trúc mạng trong đó mỗi nút mạng (được gọi là peer) vừa hoạt động như một máy khách (client) để khởi tạo yêu cầu, vừa hoạt động như một máy chủ (server) để tiếp nhận và phản hồi yêu cầu từ các nút khác. Hệ thống tạo thành một mạng phủ phân tán được xây dựng trên nền tảng hạ tầng Internet vật lý.

Trong bối cảnh của một ứng dụng trò chuyện thời gian thực, kiến trúc P2P thiết lập cơ chế hoạt động chuyên biệt:

- Mỗi người dùng vận hành một "Peer Node" độc lập, chủ động mở và lắng nghe trên một cổng TCP (TCP port) riêng biệt.
- Dữ liệu trò chuyện (tin nhắn, tệp tin) được thiết lập luồng truyền tải trực tiếp từ nút gửi đến nút nhận, bỏ qua các trạm trung chuyển.
- Khác với các hệ thống phổ thông, mạng P2P loại bỏ hoàn toàn sự hiện diện của một máy chủ lưu trữ và tiếp sức (relay) tập trung đối với nội dung thông điệp.

2.2.2. Phân loại mô hình P2P

Dựa trên mức độ phân tán, kiến trúc P2P được chia thành ba mô hình chính:

- Pure P2P: Mô hình loại bỏ hoàn toàn máy chủ trung tâm. Việc khám phá các nút mạng được thực hiện thông qua các thuật toán phân tán (như giao thức broadcast trong LAN hoặc Bảng băm phân tán DHT). Mô hình này đạt độ tin cậy cao do triệt tiêu điểm lỗi đơn, nhưng đối mặt với thách thức lớn về khả năng vượt tường lửa và độ phức tạp trong triển khai.
- Hybrid P2P: Kết hợp một máy chủ trung tâm đảm nhiệm một số chức năng điều phối hẹp (thường là xác thực và khám phá nút), trong khi dữ liệu chính vẫn được truyền tải trực tiếp giữa các nút (Ví dụ: giao thức BitTorrent).
- Bootstrap-assisted P2P (Mô hình được áp dụng trong dự án): Hệ thống triển khai một "Bootstrap Server" có nhiệm vụ duy trì sổ đăng ký chứa thông tin các nút đang hoạt động. Các nút kết nối tới Bootstrap để thực hiện thủ tục đăng ký, duy trì nhịp heartbeat và truy xuất danh sách thành viên. Đặc tính quan trọng của mô hình này là sự phân tách tuyệt đối giữa

Control Plane (Mặt phẳng điều khiển - chứa metadata, lưu trữ tạm tin nhắn ngoại tuyến tại Bootstrap) và Data Plane (Mặt phẳng dữ liệu - truyền tải tin nhắn mã hóa và tệp tin trực tiếp giữa các Peer). Mô hình này cung cấp sự cân bằng tối ưu giữa tính khả thi trong triển khai và bản chất phân tán của luồng dữ liệu.

2.3. Các thách thức của hệ thống phân tán trong chat P2P

2.3.1. Quản lý khám phá nút (Peer Discovery)

Thách thức đầu tiên trong kiến trúc phân tán là cơ chế giúp một nút mới gia nhập nhận diện được địa chỉ mạng (IP:Port) của các nút đang hoạt động. Để giải quyết bài toán này, dự án ứng dụng cơ chế Sổ đăng ký tập trung. Theo đó, Bootstrap Server đảm nhận vai trò cung cấp dữ liệu định tuyến ban đầu, trong khi hoàn toàn không can thiệp vào quá trình truyền tải dữ liệu nội dung. Mặc dù các giải pháp thay thế như DHT hay Broadcast LAN mang tính phân tán cao hơn, song cơ chế Centralized registry được lựa chọn nhờ đảm bảo độ ổn định cao trong phạm vi yêu cầu của hệ thống.

2.3.2. Quản lý trạng thái thành viên và hiện tượng Churn

Mạng P2P có tính biến động nội tại cao do các nút có thể gia nhập, chủ động rời đi hoặc mất kết nối đột ngột bất cứ thời điểm nào — hiện tượng này được định nghĩa là Churn. Quản trị sự xáo trộn này là bài toán cốt lõi thuộc lĩnh vực phát hiện lỗi. Dự án triển khai một mô hình kết hợp để xử lý Churn:

- Heartbeat định kỳ: Các nút phát tín hiệu "còn sống" tới máy chủ điều khiển theo chu kỳ 15 giây.
- Timeout Sweep: Máy chủ điều khiển quét định kỳ mỗi 5 giây, tự động loại bỏ các nút không phản hồi heartbeat trong hơn 30 giây.
- Explicit Leave: Yêu cầu các nút gửi tín hiệu rời mạng chủ đích trước khi đóng ứng dụng.
- Push Notification: Máy chủ điều khiển chủ động đẩy các thông báo trạng thái cập nhật (join/leave) đến toàn bộ các nút còn lại trong mạng.

2.3.3. Đảm bảo ngữ nghĩa giao tin (Message Delivery Semantics)

Môi trường mạng phân tán tiềm ẩn rủi ro mất mát gói tin cao. Do đó, việc xây dựng các cam kết về ngữ nghĩa giao tin là bắt buộc:

- Đối với luồng giao tiếp một-một (Chat 1-1) và truyền tệp tin, dự án áp dụng chiến lược At-least-once (Ít nhất một lần) thông qua cơ chế Xác nhận nhận (ACK) kết hợp chiến lược gửi lại lũy thừa nhằm tối đa hóa độ tin cậy.
- Đối với luồng tin nhắn toàn mạng, hệ thống chấp nhận chiến lược At-most-once (Nhiều nhất một lần) nhằm giảm thiểu áp lực băng thông. Sự trùng lặp dữ liệu trong các luồng phát sóng hoặc trò chuyện nhóm được kiểm soát triệt để ở tầng ứng dụng thông qua việc đối chiếu mã định danh thông điệp (Message ID deduplication).

2.3.4. Quản trị tính nhất quán (Consistency)

Trong các phiên trò chuyện nhóm phân tán, mỗi nút mạng lưu trữ một bản sao cục bộ về danh sách thành viên của nhóm. Khi có sự thay đổi, nút tạo nhóm sẽ phát một bản tin đồng bộ hóa. Hệ thống vận hành theo nguyên lý Eventual Consistency, chấp nhận độ trễ nhỏ trong việc đồng bộ hóa tạm thời giữa các bản sao cục bộ, hướng tới trạng thái hội tụ đồng nhất sau khi quá trình truyền tin kết thúc.

2.3.5. Bảo mật trong môi trường không tin cậy

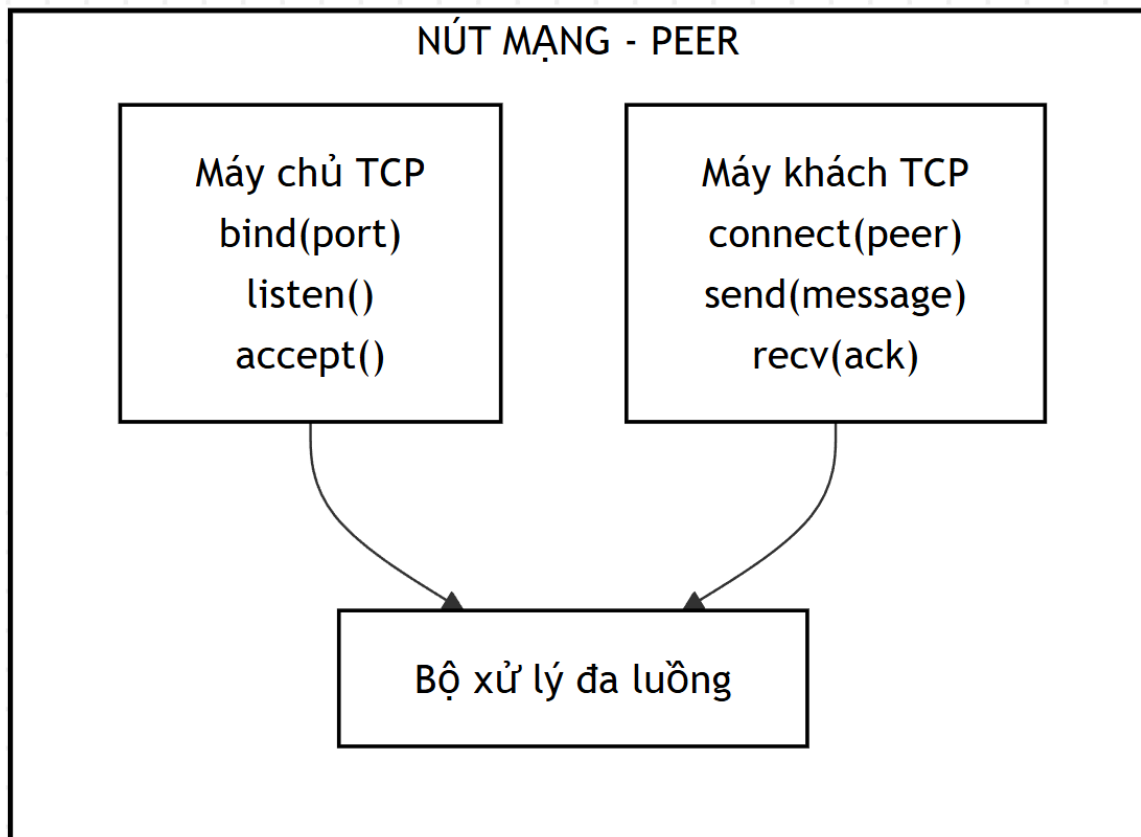
Bản chất của kiến trúc P2P là mọi nút tham gia mạng đều được coi là một thực thể không đáng tin cậy. Khác với mô hình Client-Server nơi máy chủ trung tâm được cấp quyền mặc định, hệ thống P2P yêu cầu cơ chế tự bảo vệ dữ liệu nội tại. Dự án xử lý thách thức này bằng việc triển khai kiến trúc Mã hóa đầu cuối (E2EE), sử dụng giao thức trao đổi khóa Diffie-Hellman trực tiếp giữa các nút và mã hóa dòng đối xứng. Đối với các tin nhắn chờ giao, máy chủ điều khiển chỉ lưu trữ bản mã hóa, đảm bảo tính toàn vẹn và bảo mật tuyệt đối cho nội dung đàm thoại.

2.4. Kiến trúc mạng và giao thức truyền tải

2.4.1. Mô hình TCP Client-Server trên từng peer

Đặc trưng cốt lõi của kiến trúc mạng ngang hàng đối xứng là sự xóa nhòa ranh giới giữa máy khách và máy chủ ở tầng ứng dụng. Trong dự án này, mỗi nút mạng phải đồng thời đảm nhiệm cả hai vai trò để có thể giao tiếp song hướng. Cụ thể, hệ thống tích hợp một máy chủ TCP cục bộ liên tục lắng nghe trên một cổng xác định để tiếp nhận các kết nối đến. Cùng lúc đó, một khối máy khách TCP được khởi tạo mỗi khi nút mạng cần chủ động thiết lập kết nối tới các thực

thể khác để truyền gửi thông điệp. Sự phân bổ này cho phép mọi thành viên trong mạng lưới có quyền hạn và khả năng xử lý hoàn toàn tương đương nhau.



2.4.2. Kỹ thuật đóng khung thông điệp bằng chuẩn JSON

Để đảm bảo tính nhất quán trong quá trình trao đổi dữ liệu, giao thức ứng dụng sử dụng định dạng JSON làm tiêu chuẩn cấu trúc thông điệp. Nhằm phân định ranh giới giữa các luồng dữ liệu liên tục trên kết nối TCP, hệ thống áp dụng kỹ thuật đóng khung bằng ký tự ngắt dòng.

Phương pháp tiếp cận này mang lại ưu điểm vượt trội về tính minh bạch, giúp nhà phát triển dễ dàng đọc hiểu dữ liệu thô và gỡ lỗi trong quá trình thử nghiệm. Tuy nhiên, việc sử dụng định dạng văn bản thay vì nhị phân cũng đi kèm với một sự đánh đổi nhất định về hiệu suất, làm gia tăng kích thước gói tin và hao tốn thêm tài nguyên phân tích cú pháp. Do đó, kỹ thuật này đặc biệt phù hợp với phạm vi của các dự án học thuật và mô hình thử nghiệm.

2.4.3. Mô hình xử lý đồng bộ

Hệ thống nhắn tin phân tán có đặc thù phụ thuộc rất lớn vào quá trình luân chuyển dữ liệu mạng, hay còn gọi là các tác vụ chờ nhập xuất (I/O-bound). Để ngăn chặn tình trạng thất cổ chai cục bộ, dự án ứng dụng triệt để kiến trúc đa luồng. Một luồng chính được dành riêng để duy trì vòng lặp chấp nhận kết nối, trong khi mỗi luồng phụ sẽ được sinh ra để xử lý độc lập từng phiên giao tiếp. Bên cạnh đó, hệ thống duy trì các luồng chạy ngầm để thực thi các tác vụ chu kỳ như phát tin hiệu nhịp tim, cập nhật danh sách thành viên và tiếp sức tin nhắn nhóm. Nhằm đảm bảo tính toàn vẹn dữ liệu khi có nhiều luồng cùng truy xuất, cơ chế khóa đồng bộ được áp dụng chặt chẽ trên các vùng nhớ dùng chung như danh sách nút mạng, kho lưu trữ khóa bảo mật và cấu hình nhóm.

2.5. Các thuật toán liên quan

2.5.1. Trao đổi khóa bảo mật Diffie-Hellman

Thuật toán Diffie-Hellman đóng vai trò nền tảng trong việc thiết lập kênh liên lạc an toàn. Thuật toán này cho phép hai thực thể kiến tạo một khóa bí mật chung trên một môi trường truyền tải công cộng mà không cần chia sẻ trực tiếp khóa đó qua mạng.

Dựa trên tính chất toán học của logarit rời rạc, quy trình được vận hành với một số nguyên tố lớn p và phần tử sinh g như sau:

- Nút A chọn một khóa riêng tư ngẫu nhiên a , tính toán khóa công khai:
$$A = g^a \bmod p$$
- Nút B chọn một khóa riêng tư ngẫu nhiên b , tính toán khóa công khai:
$$B = g^b \bmod p$$
- Nút A tính toán khóa bí mật chung: $K = B^a \bmod p$
- Nút B tính toán khóa bí mật chung: $K = A^b \bmod p$

Sự hội tụ của thuật toán được chứng minh qua phương trình:

$$B^a \bmod p = (g^b)^a \bmod p = g^{ab} \bmod p = (g^a)^b \bmod p = A^b \bmod p$$

Trong khuôn khổ dự án, hệ thống sử dụng số nguyên tố chuẩn độ dài 512-bit. Khóa bí mật chung sau khi thiết lập sẽ được băm qua hàm SHA-256 để triết xuất thành một khóa đối xứng 256-bit. Quá trình bắt tay mật mã này diễn ra

hoàn toàn trực tiếp giữa hai nút, thể hiện rõ tính chất phi tập trung trong việc kiến tạo bảo mật mà không cần sự can thiệp của máy chủ trung gian.

2.5.2. Mã hóa luồng đối xứng

Sau khi thiết lập khóa chung, toàn bộ dữ liệu giao tiếp được bảo vệ bằng cơ chế mã hóa luồng, dựa trên nguyên lý khởi tạo chuỗi khóa từ hàm băm SHA-256.

Quá trình sinh chuỗi khóa được thực hiện theo công thức:

$$Keystream[i] = SHA - 256(Key || IV || counter)[0... 31]$$

Bản mã cuối cùng được tạo ra bằng phép toán logic XOR giữa bản rõ và chuỗi khóa:

$$Ciphertext = Plaintext \oplus Keystream$$

Hệ thống sử dụng một Véc-tơ khởi tạo (IV) độ dài 16 byte ngẫu nhiên cho mỗi luồng thông điệp nhằm ngăn chặn các cuộc tấn công phân tích mẫu. Đồng thời, biến đếm được tăng dần theo từng khối 32 byte để mở rộng chuỗi khóa đối với các tệp dữ liệu dung lượng lớn. Đặc tính đối xứng của phép toán XOR cho phép sử dụng chung một hàm thuật toán cho cả quá trình mã hóa lẫn giải mã, tối ưu hóa đáng kể tài nguyên tính toán.

2.5.3. Giao thức lan truyền tin đồn

Đối với bài toán phát sóng thông điệp diện rộng, dự án áp dụng giao thức lan truyền tin đồn. Tương tự như sự lây lan của mầm bệnh, một nút khi nhận được thông điệp mới sẽ xử lý nội dung, sau đó chủ động sao chép và chuyển tiếp thông điệp đó đến các nút lân cận trong danh sách định tuyến.

Để kiểm soát băng thông và tránh hiện tượng bão mạng, thuật toán áp dụng hai cơ chế ràng buộc:

- Lọc trùng lặp: Hệ thống lưu vết mã định danh của các thông điệp đã xử lý để từ chối các bản sao thừa.
- Thời gian sống: Mỗi thông điệp phát sóng được gán một vòng đời tối đa (trong dự án là 3 bước nhảy). Chỉ số này sẽ giảm dần khi đi qua mỗi nút trung gian và thông điệp sẽ bị hủy khi chỉ số bằng không.

2.5.4. Lưu trữ và chuyển tiếp

Kỹ thuật lưu trữ và chuyển tiếp được áp dụng để xử lý tình trạng mất kết nối cục bộ. Khi một nút mạng không thể gửi trực tiếp tin nhắn đến đích (do đối tác đang ngoại tuyến), tin nhắn sẽ được định tuyến lên máy chủ trung gian. Máy chủ này đóng vai trò như một bộ đệm, lưu trữ tạm thời gói tin vào hàng đợi và sẽ tự động phân phối lại ngay khi nút đích tái kết nối mạng. Điểm cốt lõi về mặt an toàn là toàn bộ thông điệp đã được mã hóa đầu cuối từ trước; do đó, máy chủ chỉ tiếp nhận và lưu trữ bản mã, hoàn toàn không có khả năng đọc hiểu hay can thiệp vào bản rõ.

2.5.5. Chiến lược thử lại với độ trễ tăng lũy thừa

Nhằm đảm bảo độ tin cậy trong các luồng giao tiếp quan trọng (trò chuyện cá nhân, truyền tệp), dự án tích hợp chiến lược gửi lại tự động với độ trễ tăng lũy thừa. Khi một gói tin gửi đi không nhận được tín hiệu xác nhận, hệ thống sẽ tự động thử lại nhiều lần. Khoảng thời gian chờ giữa các lần thử sẽ tăng theo cấp số nhân (ví dụ: 1 giây, 2 giây, 4 giây) trước khi hệ thống đánh giá đường truyền đã đứt gãy và chuyển sang cơ chế lưu trữ tạm thời. Kỹ thuật này giúp hệ thống tránh được hiện tượng quá tải bầy đàn khi mạng gặp sự cố chập chờn tạm thời.

2.5.6. Phát hiện lỗi qua tín hiệu heartbeat

Dự án vận hành một cơ chế phát hiện lỗi dựa trên thời gian chờ. Các nút mạng có nghĩa vụ phải liên tục báo cáo trạng thái hoạt động thông qua việc gửi tín hiệu nhịp tim mỗi 15 giây về máy chủ điều khiển. Ở chiều ngược lại, một cơ chế quét tự động trên máy chủ sẽ hoạt động với chu kỳ 5 giây để rà soát thời gian báo cáo cuối cùng. Mọi nút mạng không phát sinh tín hiệu vượt quá 30 giây sẽ bị kết luận là đã mất kết nối. Ngay lập tức, máy chủ sẽ kích hoạt tiến trình đẩy thông báo cập nhật sự cố đến toàn bộ các thành viên còn lại để đồng bộ hóa trạng thái mạng lưới.

2.6. Mô hình phân tán Control Plane và Data Plane

Một trong những tư tưởng kiến trúc quan trọng nhất được áp dụng trong dự án là việc tách bạch hoàn toàn chức năng điều phối hệ thống ra khỏi quá trình luân chuyển dữ liệu thực tế. Cấu trúc này được phân định rõ ràng như sau:

Mặt phẳng	Thành phần phụ trách	Chức năng nghiệp vụ	Loại dữ liệu xử lý
-----------	----------------------	---------------------	--------------------

Mặt phẳng điều khiển	Máy chủ trung tâm (Bootstrap)	Đăng ký mạng, giám sát nhịp tim, khám phá thành viên, quản lý hàng đợi ngoại tuyến.	Siêu dữ liệu (Địa chỉ IP, Cổng, Định danh).
Mặt phẳng dữ liệu	Các nút mạng ngang hàng	Truyền tải tin nhắn mã hóa cá nhân, phát sóng, trò chuyện nhóm và truyền tệp tin.	Nội dung thông điệp bản mã.

Bảng 2.2: Phân tích vai trò giữa các mặt phẳng hệ thống

Sự phân tách này mang lại ba lợi ích kỹ thuật to lớn. Thứ nhất, nó gia tăng sức chịu đựng của hệ thống: ngay cả khi máy chủ điều khiển ngừng hoạt động, các nút mạng đã kết nối từ trước vẫn có thể tiếp tục trò chuyện trực tiếp mà không bị gián đoạn. Thứ hai, máy chủ điều khiển được giải phóng hoàn toàn khỏi áp lực băng thông do không phải làm trạm trung chuyển dữ liệu. Cuối cùng, mô hình này thiết lập một ranh giới bảo mật vững chắc, đảm bảo máy chủ trung tâm bị cô lập hoàn toàn khỏi nội dung giao tiếp nhạy cảm của người dùng.

Chương 3: Phân tích và thiết kế hệ thống

3.1. Phân tích yêu cầu

3.1.1. Bối cảnh và bài toán đặt ra

Các hệ thống trò chuyện truyền thống theo mô hình Client-Server thường định tuyến toàn bộ luồng dữ liệu thông qua một máy chủ trung tâm, dẫn đến các rủi ro về điểm lỗi đơn và nguy cơ xâm phạm quyền riêng tư. Trong khuôn khổ nghiên cứu về hệ thống phân tán, dự án ELITE P2P CHAT được đề xuất nhằm chứng minh tính khả thi của mô hình ứng dụng trò chuyện thời gian thực đáp ứng ba tiêu chí cốt lõi:

- Mặt phẳng dữ liệu (Data Plane): Mọi nội dung tin nhắn và tệp tin phải được luân chuyển trực tiếp giữa các nút mạng (Peer) mà không qua trung gian.
- Mặt phẳng điều khiển (Control Plane): Được hỗ trợ bởi một máy chủ Bootstrap để quản lý trạng thái trực tuyến và hàng đợi ngoại tuyến, nhưng máy chủ này tuyệt đối không có khả năng giải mã nội dung thông điệp.

3. Vai trò đối xứng: Mỗi nút mạng tham gia phải đồng thời thực thi năng lực của cả máy khách và máy chủ, tuân thủ đúng nguyên lý cốt lõi của mạng ngang hàng.

3.1.2. Yêu cầu chức năng

Hệ thống được chia thành ba nhóm chức năng chính:

Nhóm A — Quản lý thành viên:

- Đăng ký nút mạng: Ghi nhận thông tin (Tên, IP, Cổng) vào sổ đăng ký và thông báo sự hiện diện cho toàn mạng.
- Duy trì nhịp tim (Heartbeat): Liên tục cập nhật trạng thái hoạt động; tự động loại bỏ các nút không phản hồi tín hiệu quá 30 giây.
- Khám phá mạng: Cung cấp danh sách các nút đang trực tuyến (loại trừ chính nút gửi yêu cầu).
- Thông báo sự kiện mạng: Máy chủ chủ động đẩy thông báo khi có thành viên gia nhập hoặc rời đi.

Nhóm B — Truyền tải mạng ngang hàng (Data Plane):

- Chat 1-1 E2EE: Thiết lập khóa bảo mật Diffie-Hellman trực tiếp giữa hai nút; truyền tải bản mã mà không qua Bootstrap.
- Độ tin cậy tầng ứng dụng: Áp dụng cơ chế Xác nhận nhận (ACK) và thử lại khi gửi lỗi.
- Lưu trữ và chuyển tiếp: Máy chủ Bootstrap đóng vai trò lưu trữ tạm các gói tin đã mã hóa (blind blob) khi nút đích ngoại tuyến.
- Phát sóng (Broadcast): Lan truyền thông điệp toàn mạng bằng thuật toán Epidemic Flooding, có giới hạn bước nhảy và lọc trùng lặp.
- Trò chuyện nhóm: Quản lý trạng thái nhóm phân tán và truyền tin tiếp sức đa điểm.
- Truyền tệp tin E2EE: Cho phép truyền tệp tin mã hóa với dung lượng giới hạn dưới 2MB.

Nhóm C — Giao diện người dùng: Cung cấp cả giao diện dòng lệnh (CLI) phục vụ kiểm thử và giao diện đồ họa (GUI Tkinter) hỗ trợ trải nghiệm người dùng đa luồng.

3.1.3. Yêu cầu phi chức năng

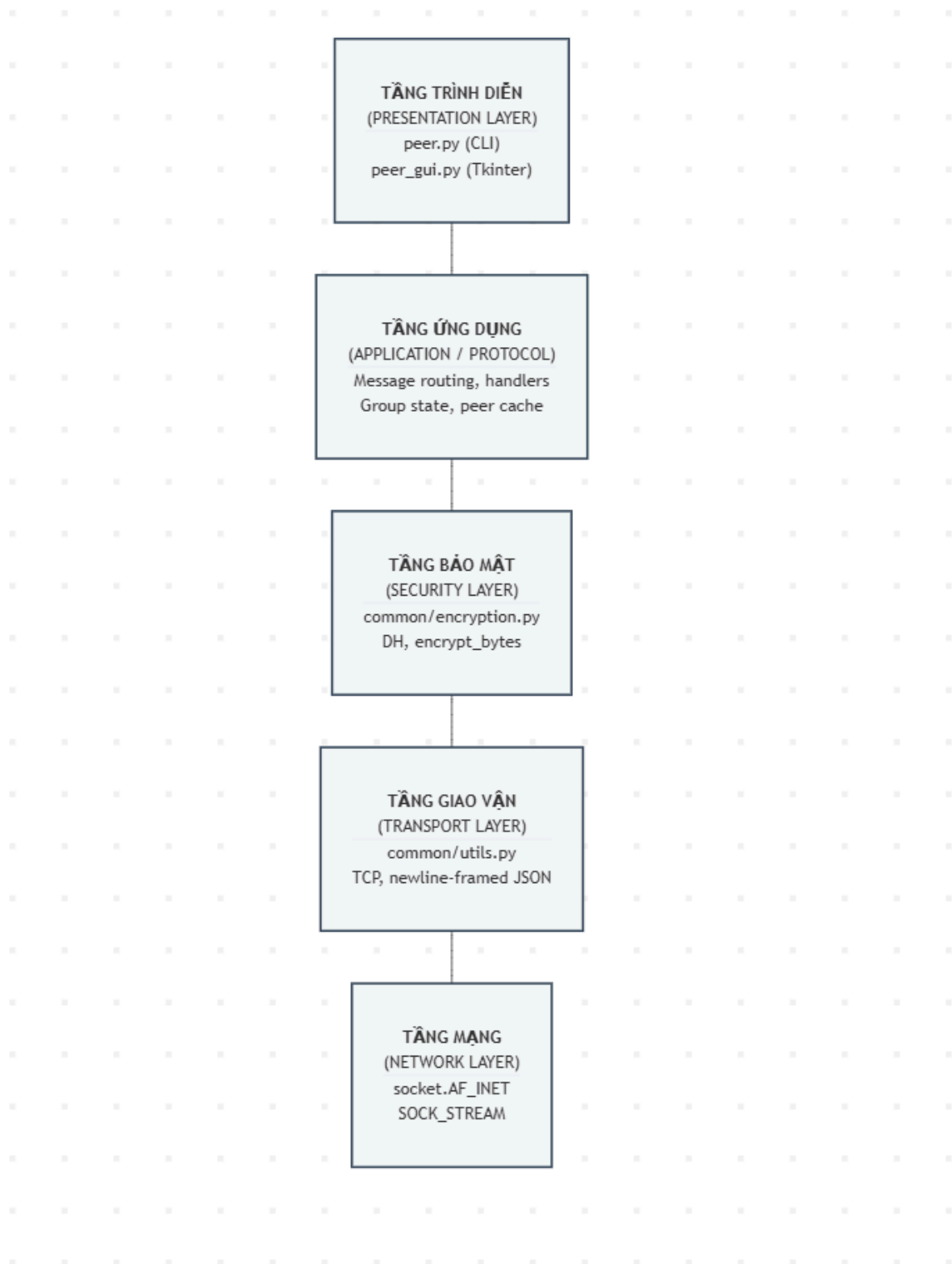
Dự án được xây dựng dựa trên các tiêu chí ràng buộc về chất lượng kỹ thuật:

- Tính phân tán toàn diện: 100% luồng giao tiếp 1-1 và truyền tệp tin phải sử dụng kết nối TCP trực tiếp.
- Bảo mật tuyệt đối: Máy chủ Bootstrap không được phép tiếp cận khóa chung của các cuộc hội thoại cá nhân.
- Độ tin cậy luồng truyền: Tích hợp cơ chế ACK với tối đa 3 lần thử lại tự động; cơ chế thử lại lũy thừa để chống quá tải.
- Sức chịu đựng xáo trộn mạng (Churn): Đảm bảo hệ thống duy trì hoạt động ổn định thông qua cơ chế cập nhật nhịp tim và hàng đợi chờ khi các nút liên tục ra/vào mạng.

3.2. Kiến trúc tổng thể

3.2.1. Mô hình phân tầng

Kiến trúc hệ thống được phân rã thành bốn tầng chức năng riêng biệt, giúp tăng cường khả năng bảo trì và giảm thiểu sự phụ thuộc giữa các khối logic.



3.2.2. Mô hình Bootstrap-assisted P2P

Dự án triển khai một mô hình kết hợp (Hybrid) có kiểm soát. Hệ thống không hoạt động theo P2P thuần túy, cũng không đi theo lối mòn Client-Server. Trong mô hình này, máy chủ Bootstrap nắm giữ điểm lỗi đơn đối với khâu khám phá

mạng, nhưng hoàn toàn bị loại bỏ khỏi mặt phẳng truyền tải nội dung. Điều này đảm bảo quyền riêng tư và khả năng mở rộng không giới hạn cho luồng giao tiếp P2P.

3.2.3. Tính đối xứng của nút mạng

Trái ngược với kiến trúc truyền thống phân tách máy chủ và máy khách, mỗi nút trong hệ thống ELITE P2P CHAT được thiết kế đồng nhất và đối xứng. Mỗi nút đều tích hợp một cổng lắng nghe (Server socket) chạy ngầm liên tục và một khối khởi tạo kết nối (Client socket) hoạt động linh hoạt, cho phép truyền tin trực tiếp, chuyển tiếp thông báo nhóm và nhận tệp tin mà không cần phân loại phân cấp.

3.3 Thiết kế Bootstrap Server

3.3.1. Phân định ranh giới trách nhiệm

Máy chủ Bootstrap được thiết kế với vai trò tối giản nhưng mang tính định chuẩn cho toàn mạng. Trách nhiệm chính bao gồm: duy trì sổ đăng ký các nút, loại bỏ các nút chết thông qua kiểm tra nhịp tim, cung cấp thông tin định tuyến (IP:Port), lưu trữ hộp thư chờ cho các nút ngoại tuyến và đẩy thông báo sự kiện mạng. Đổi lại, hệ thống nghiêm cấm Bootstrap thực hiện các hành vi như làm trạm trung chuyển (relay) thông điệp thông thường, tham gia trao đổi khóa Diffie-Hellman hay lưu trữ dữ liệu lịch sử dài hạn.

3.3.2. Vòng đời kết nối và Xử lý nghiệp vụ

Nhằm tối ưu hóa tài nguyên và tránh tình trạng giữ hàng nghìn kết nối treo, Bootstrap vận hành theo mô hình kết nối TCP vòng đời ngắn. Khi có nhu cầu, các nút mạng mở kết nối tới Bootstrap, gửi một bản tin JSON, nhận phản hồi và đóng kết nối ngay lập tức.

Luồng xử lý các nhóm nghiệp vụ chính của Bootstrap diễn ra như sau:

- Ghi danh: Xác thực thông tin, nếu nút mới xuất hiện sẽ thêm vào bộ nhớ và phát tin hiệu peer_joined thông báo cho các nút còn lại.
- Khám phá (Get Peers): Trích xuất danh sách tất cả các nút hiện tại, loại trừ địa chỉ của chính nút gửi yêu cầu để tránh rác luồng.
- Kiểm soát nhịp tim (Heartbeat & Sweep): Cập nhật bộ đếm thời gian mỗi khi nhận được tin hiệu. Song song đó, một luồng quét ngầm chạy mỗi 5

giây sẽ gỡ bỏ các nút bị "treo" (quá 30 giây không hồi đáp) và kích hoạt lệnh đẩy thông báo peer_left.

- Lưu trữ chờ (Store & Forward): Tiếp nhận bản mã từ nút gửi, kiểm tra tính hợp lệ của định dạng JSON và đẩy vào hàng đợi chờ (dựa theo IP:Port của nút đích) mà không can thiệp vào cấu trúc bản mã. Khi nút đích tái kết nối mạng và gửi yêu cầu, Bootstrap sẽ giải phóng toàn bộ hộp thư thành một gói (batch) và xóa sạch hàng đợi theo chuẩn ngữ nghĩa *at-most-once* để tránh rò rỉ bộ nhớ.
- Thông báo chủ động: Thay vì bắt các nút mạng phải liên tục thăm dò định kỳ, Bootstrap sử dụng cơ chế đẩy sự kiện. Khi danh sách thành viên thay đổi, Bootstrap sinh ra các luồng phụ, chủ động mở kết nối đảo ngược tới cổng của từng nút để gửi thông báo, qua đó đảm bảo trạng thái mạng được cập nhật theo thời gian thực.

3.3.3. Chi tiết từng handler

handle_register(conn, msg, addr)

Bước	Hành vi
1	Validate ip, port, name
2	Tìm peer theo (ip, port)
3a	Đã tồn tại → cập nhật name, last_heartbeat
3b	Mới → append peers, is_new=True
4	Trả {status: success}
5	Nếu is_new → _notify_all_peers(peer_joined)

handle_get_peers(conn, msg)

- Input: ip, port của requester
- Output: peer_list — tất cả peer trừ requester (tránh peer tự chat với chính mình qua list)

handle_heartbeat(conn, msg)

- Cập nhật last_heartbeat = time.time()
- Trả lỗi nếu peer chưa register

`handle_leave(conn, msg)`

- Xóa peer khỏi peers
- Push `peer_left` tới các peer còn lại

`handle_store_forward(conn, msg)`

- Input: `target_ip`, `target_port`, `message` (dict)
- Validate message là dict
- Append vào `pending_messages[(target_ip, target_port)]`
- Không validate/giải mã nội dung — coi như opaque blob

`handle_get_pending(conn, msg)`

- `pop(key, [])` — giao toàn bộ queue và xóa (at-most-once từ phía queue)
- Trả `pending_batch` với `messages: [...]`

3.4. Thiết kế Peer Node

3.4.1. Cấu trúc dữ liệu cốt lõi

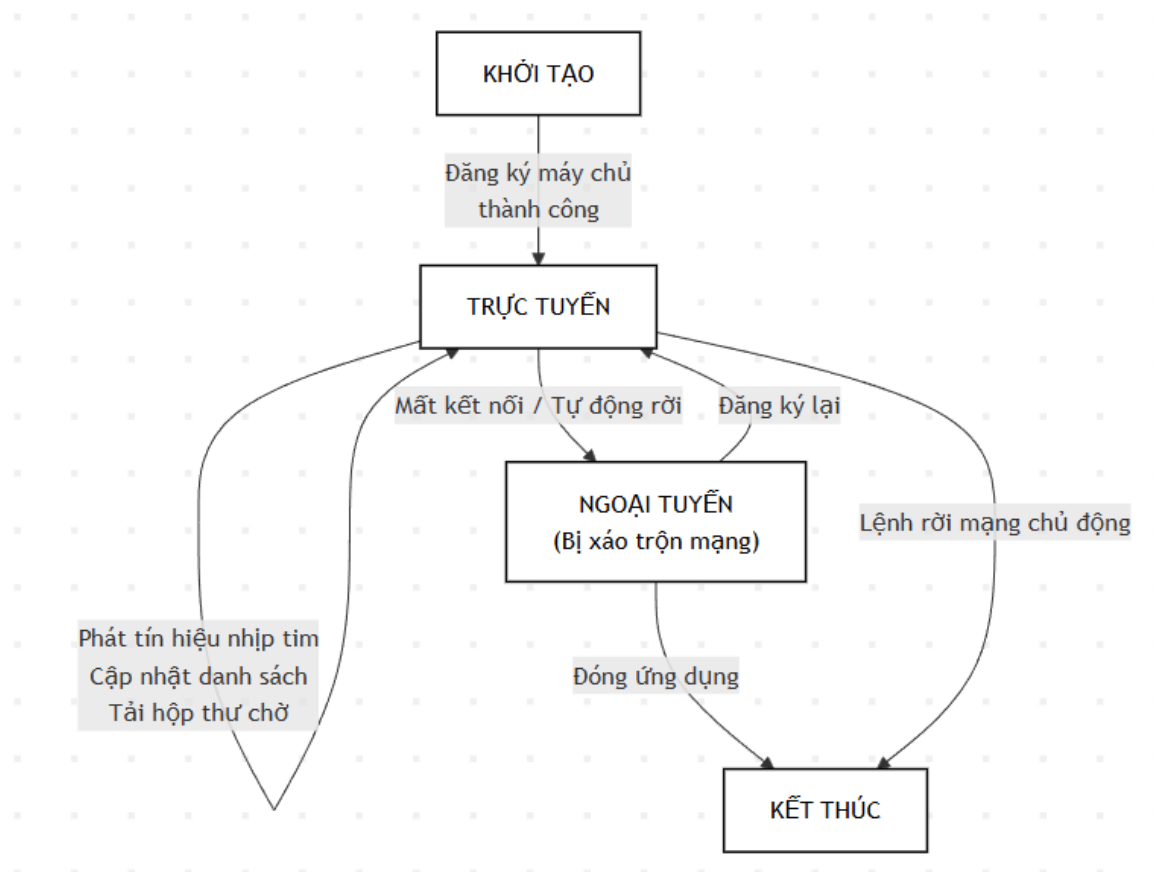
Kiến trúc nội tại của mỗi nút mạng được triển khai thông qua một lớp đối tượng toàn diện, có nhiệm vụ lưu trữ và quản lý toàn bộ trạng thái hoạt động cục bộ. Các thuộc tính cốt lõi của đối tượng này được chia thành nhiều nhóm nghiệp vụ. Nhóm định danh bao gồm địa chỉ IP, cổng giao tiếp và tên hiển thị mạng. Nhóm kết nối lưu trữ thông tin định tuyến của máy chủ điều khiển trung tâm và duy trì một bộ đệm chứa danh sách các thành viên đang hoạt động.

Về mặt an toàn thông tin, mỗi nút tự quản lý một kho lưu trữ các khóa bảo mật đầu cuối (được ánh xạ theo từng cặp IP và cổng của đối tác) nhằm đảm bảo quá trình mã hóa không bị can thiệp. Để hỗ trợ các tính năng phân tán, nút mạng duy trì một tập hợp các mã định danh thông điệp nhằm lọc bỏ các gói tin phát sóng trùng lặp, đồng thời sử dụng một cấu trúc từ điển để lưu trữ trạng thái của các nhóm trò chuyện cục bộ. Toàn bộ các vùng nhớ dùng chung này đều được bảo vệ bởi cơ chế khóa đồng bộ nhằm đảm bảo an toàn luồng. Cuối cùng, vòng đời của nút mạng được kiểm soát thông qua các cờ trạng thái hoạt động và một cổng giao tiếp luôn trong trạng thái lắng nghe kết nối.

Thuộc tính	Kiểu	Vai trò
ip, port, name	str, int, str	Định danh mạng
bootstrap_addr	tuple	Địa chỉ control plane
peer_list	list	Cache discovery
shared_keys	dict	E2EE per (ip, port)
seen_messages	set	Đeup broadcast/group
groups	dict	State nhóm local
online, running	bool	Vòng đời
lock	Lock	Đồng bộ shared state
server_socket	socket	TCP listener

3.4.2. Máy trạng thái vòng đời

Vòng đời hoạt động của một nút mạng được mô hình hóa dưới dạng một máy trạng thái hữu hạn, bao gồm bốn trạng thái chính: Khởi tạo, Trục tuyến, Ngoại tuyến và Kết thúc. Sự chuyển đổi giữa các trạng thái phản ánh khả năng tự phục hồi của hệ thống trước hiện tượng xáo trộn mạng.



Giai đoạn Khởi tạo diễn ra khi nút mạng bắt đầu nạp cấu hình và gửi yêu cầu đăng ký lên máy chủ. Khi quá trình này thành công, hệ thống chuyển sang trạng thái Trực tuyến. Tại đây, nút mạng liên tục thực hiện các chu trình chạy ngầm như phát tín hiệu duy trì, đồng bộ danh sách thành viên và kiểm tra hộp thư chờ. Nếu xảy ra sự cố rớt mạng hoặc mô phỏng xáo trộn, nút mạng rơi vào trạng thái Ngoại tuyến nhưng vẫn duy trì các tiến trình cốt lõi để sẵn sàng đăng ký lại và khôi phục trạng thái Trực tuyến. Khi nhận được lệnh thoát chủ động từ người dùng, hệ thống sẽ giải phóng tài nguyên và chuyển sang trạng thái Kết thúc.

3.4.3. Kiến trúc xử lý đa luồng

Do đặc thù của hệ thống nhắn tin phân tán là liên tục chờ đợi các tín hiệu vào/ra trên môi trường mạng, bài toán thất cổ chai hiệu năng được giải quyết triệt để thông qua mô hình lập trình đa luồng. Cấu trúc luồng được phân chia theo từng nhóm tác vụ cụ thể:

- Luồng giao diện và lắng nghe: Luồng chính của chương trình chịu trách nhiệm vận hành vòng lặp giao diện người dùng (dòng lệnh hoặc đồ họa)

để tiếp nhận tương tác. Song song đó, một luồng máy chủ chạy độc lập, liên tục vòng lặp chấp nhận các kết nối mạng gửi đến.

- Luồng xử lý phiên liên lạc: Mỗi khi có một kết nối mới được thiết lập, hệ thống lập tức khởi tạo một luồng phụ để chuyên trách xử lý gói tin đầu vào. Điều này đảm bảo một thông điệp có dung lượng lớn không làm đình trệ toàn bộ hệ thống.
- Luồng duy trì và đồng bộ (Chạy ngầm): Hệ thống cấp phát các luồng nền với chu kỳ hoạt động cố định. Một luồng phát tín hiệu hoạt động mỗi 15 giây để báo cáo máy chủ, trong khi luồng khác chạy mỗi 10 giây để đồng bộ hóa danh sách thành viên và truy xuất các thông điệp ngoại tuyến.
- Luồng chuyển tiếp đa điểm: Đối với các tác vụ đòi hỏi thời gian thực thi lâu như phát sóng toàn mạng hay tiếp sức tin nhắn nhóm, hệ thống sử dụng các luồng chuyển tiếp độc lập để đẩy gói tin đi mà không làm treo tiến trình giao tiếp hiện tại.

Luồng	Hàm	Chu kỳ/Kích hoạt
Main	cli_loop()/GUI mainloop	Input người dùng
Server	start_server()	Liên tục accept()
Per-connection	handle_incoming()	Mỗi TCP đến
Heartbeat	heartbeat_loop()	15 giây
Discovery	update_peer_list_loop()	10 giây (+fetch pending)
Forward	threading.Thread(send_direct_message)	Broadcast/group relay
Churn (opt)	churn_loop()	online/offline cycle

3.4.4. Bộ định tuyến thông điệp trung tâm

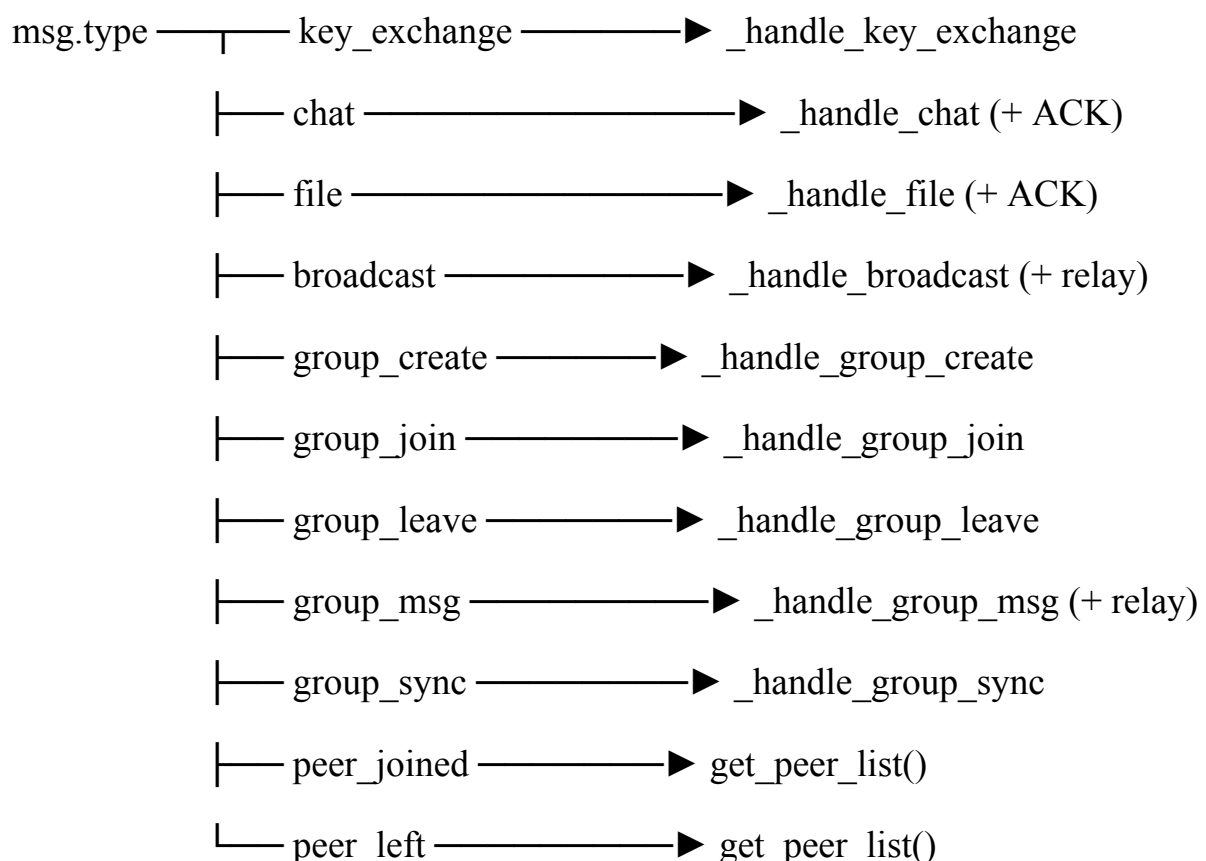
Mọi luồng dữ liệu tiến vào nút mạng, dù từ máy chủ điều khiển hay từ các nút ngang hàng khác, đều được hội tụ và phân giải tại một bộ định tuyến thông điệp trung tâm. Thành phần này đóng vai trò như một trạm phân loại, đọc trường kiểu dữ liệu (type) của gói tin và điều phối luồng xử lý tới các hàm nghiệp vụ tương ứng.

Quá trình điều phối được phân mảnh thành các nhánh nghiệp vụ cụ thể. Nhóm bảo mật xử lý các gói tin thỏa thuận khóa chung. Nhóm giao tiếp trực tiếp phụ

trách bóc tách bản mã của tin nhắn cá nhân và tệp tin, đồng thời kích hoạt tín hiệu xác nhận (ACK). Nhóm truyền tin diện rộng xử lý các bản tin phát sóng, kết hợp kiểm tra tính trùng lặp và giảm trừ vòng đời gói tin trước khi tiếp tục lan truyền. Nhóm nghiệp vụ đa điểm phụ trách toàn bộ vòng đời của cuộc trò chuyện nhóm, từ việc khởi tạo, đồng bộ thành viên đến tiếp sức thông điệp. Cuối cùng, nhóm sự kiện mạng chịu trách nhiệm tái cấu trúc lại bộ đệm danh sách khi nhận được thông báo có thành viên mới tham gia hoặc vừa rời đi.

Một điểm tinh tế trong thiết kế của bộ định tuyến này là khả năng xử lý linh hoạt đối tượng kết nối. Khi xử lý các tin nhắn cá nhân nhận được từ hộp thư chờ của máy chủ trung tâm, bộ định tuyến sẽ nhận diện kết nối là rỗng và chủ động bỏ qua bước gửi tín hiệu xác nhận ngược, do đối tác gửi tin thực tế không còn giữ phiên liên lạc trực tiếp ở thời điểm đó.

`_process_incoming_message(conn, addr, msg)` - bộ định tuyến trung tâm tại mỗi peer:



`conn` có thể `None` khi xử lý pending từ Bootstrap, không gửi ACK qua `connection pending`.

3.5. Thiết kế giao thức ứng dụng

3.5.1. Quy ước đóng khung thông điệp

Giao thức tầng ứng dụng của hệ thống sử dụng định dạng chuỗi JSON làm tiêu chuẩn cấu trúc thông điệp, kết hợp với ký tự ngắt dòng đóng vai trò là cờ phân định ranh giới giữa các gói tin liên tiếp trên cùng một luồng TCP.

Về mặt vận hành, quá trình truyền gửi yêu cầu hệ thống chuyển đổi từ điển dữ liệu thành chuỗi JSON, nối thêm ký tự ngắt dòng và mã hóa theo chuẩn UTF-8 trước khi đẩy vào đường truyền mạng. Ở chiều ngược lại, tiến trình nhận sẽ đọc luồng dữ liệu cho đến khi phát hiện ký tự ngắt dòng, tiến hành cắt bỏ khoảng trắng thừa và giải mã chuỗi văn bản ngược lại thành đối tượng dữ liệu nội tại.

Thiết kế này mang lại sự đơn giản và dễ dàng trong công tác gỡ lỗi, tuy nhiên cũng tồn tại những giới hạn kỹ thuật nhất định. Do cơ chế đọc theo từng dòng, hệ thống không cho phép các trường dữ liệu chứa ký tự xuống dòng thô. Đặc biệt, đối với tác vụ truyền tệp tin, toàn bộ nội dung nhị phân bắt buộc phải được mã hóa sang định dạng Base64 và nhúng trực tiếp vào một trường duy nhất bên trong cấu trúc JSON. Đây chính là nguyên nhân hệ thống phải thiết lập giới hạn dung lượng tệp tin ở mức 2MB nhằm ngăn chặn hiện tượng cạn kiệt bộ nhớ trong quá trình phân tích cú pháp chuỗi.

3.5.2. Đặc tả thông điệp tại Mặt phẳng điều khiển

Mặt phẳng điều khiển phụ trách các giao dịch siêu dữ liệu giữa nút mạng và máy chủ Bootstrap.

Kiểu thông điệp (type)	Các trường dữ liệu bắt buộc	Ý nghĩa và Mô tả nghiệp vụ	Định dạng phản hồi dự kiến
register	ip (str), port (int), name (str)	Gửi yêu cầu đăng ký nút mạng mới vào hệ thống.	Trạng thái thành công hoặc báo lỗi kèm theo mô tả chi tiết.

get_peers	ip (str), port (int)	Truy xuất danh sách toàn bộ các thành viên đang hoạt động.	Gói tin chứa danh sách các nút mạng (đã loại trừ định danh của nút gửi yêu cầu).
heartbeat	ip (str), port (int)	Tín hiệu duy trì trạng thái hoạt động định kỳ.	Không yêu cầu phản hồi trừ khi nút mạng chưa đăng ký.
leave	ip (str), port (int)	Lệnh thông báo chủ động rút khỏi mạng lưới.	Không yêu cầu phản hồi.

Bảng 3.1 trình bày chi tiết cấu trúc của các thông điệp cốt lõi phục vụ quá trình quản lý thành viên.

Đối với bài toán xử lý thông điệp ngoại tuyến, hệ thống định nghĩa một nhóm gói tin chuyên biệt phục vụ cơ chế lưu trữ và chuyển tiếp, được mô tả trong Bảng 3.2.

Kiểu thông điệp (type)	Các trường dữ liệu bắt buộc	Ý nghĩa và Mô tả nghiệp vụ
store_forward	target_ip, target_port, message	Yêu cầu máy chủ lưu trữ tạm một gói tin toàn vẹn (đã được mã hóa đầu cuối) để chờ chuyển phát đến nút đích.
get_pending	ip, port	Lệnh yêu cầu máy chủ giải phóng toàn bộ hộp thư chờ của nút hiện tại.
pending_batch	messages (mảng đối tượng)	Gói tin phản hồi từ máy chủ, chứa danh sách toàn bộ các thông điệp đã lưu trữ.

Bảng 3.2: Đặc tả thông điệp hàng đợi ngoại tuyến

Bên cạnh đó, hệ thống sử dụng các thông điệp đẩy chủ động (Push notifications) bao gồm peer_joined và peer_left. Các gói tin này do máy chủ trung tâm khởi tạo và đẩy đến các nút mạng, chứa thông tin chi tiết (tên, IP, cổng) của thành viên vừa gia nhập hoặc vừa rời đi.

3.5.3. Đặc tả thông điệp tại Mặt phẳng dữ liệu

Mặt phẳng dữ liệu đảm nhiệm quá trình luân chuyển nội dung hội thoại trực tiếp giữa các nút mạng. Đây là khu vực áp dụng triệt để các giao thức bảo mật mật mã học.

Kiểu thông điệp	Các trường dữ liệu cốt lõi	Ràng buộc và Mô tả nghiệp vụ
key_exchange	from, from_ip, from_port, pub_key	Khởi tạo tiến trình bắt tay Diffie-Hellman. Chứa khóa công khai của nút gửi.
key_exchange_reply	from, pub_key	Phản hồi tiến trình bắt tay. Phải được gửi trả trên chính kết nối TCP đã khởi tạo.
chat	from, from_ip, from_port, ciphertext, iv, msg_id	Truyền tải bản mã tin nhắn. Yêu cầu Véc-tơ khởi tạo (IV) 16 byte chuẩn Base64 và mã định danh duy nhất (UUID) để phục vụ xác nhận.
file	Các trường của chat kèm theo filename	Chuyển phát tệp tin. Toàn bộ tải trọng tệp được mã hóa đối xứng và chuyển đổi sang chuỗi Base64.
ack	msg_id	Tín hiệu xác nhận đã nhận thành công. Trường định danh phải khớp tuyệt đối với gói tin gốc.

Bảng 3.3: Đặc tả thông điệp trao đổi khóa và bảo mật cá nhân

Đối với các luồng giao tiếp diện rộng mang tính phân tán, thông điệp được thiết kế để hỗ trợ định tuyến đa điểm và kiểm soát vòng đời của gói tin, được trình bày chi tiết tại Bảng 3.4.

Kiểu thông điệp	Các trường dữ liệu cốt lõi	Ràng buộc và Mô tả nghiệp vụ
-----------------	----------------------------	------------------------------

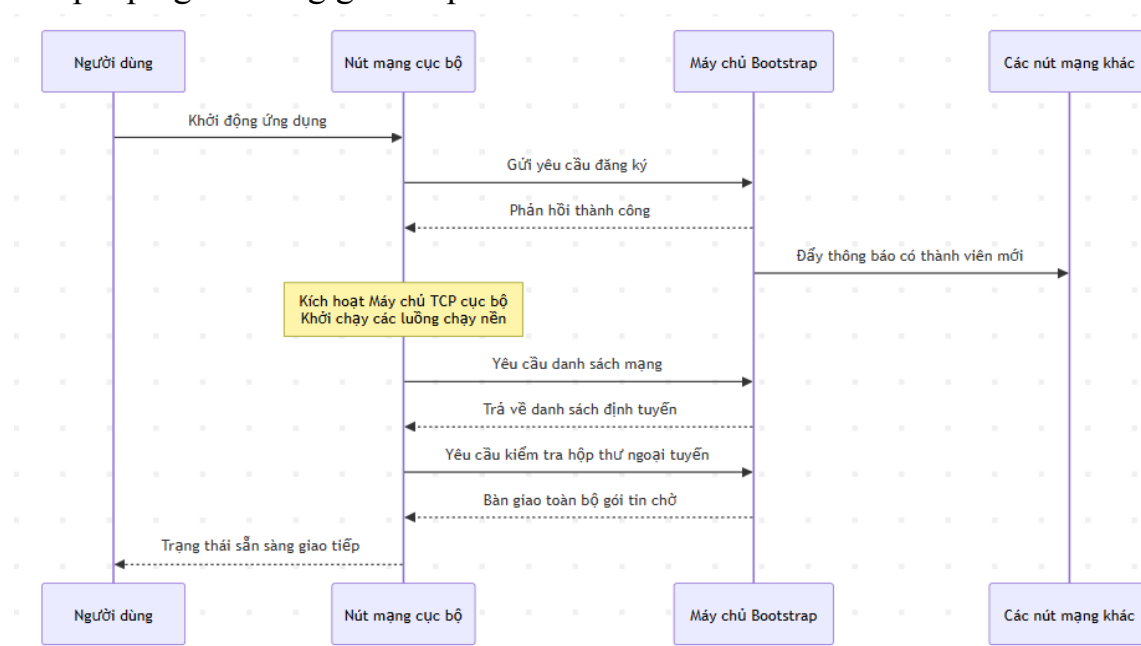
broadcast	content, msg_id, ttl	Lan truyền toàn mạng. msg_id dùng để lọc trùng lặp; chỉ số thời gian sống (ttl) quy định số bước nhảy tối đa (mặc định là 3). Nội dung để ở dạng bản rõ phục vụ mục đích kiểm thử.
group_msg	group_name, content, msg_id	Truyền tải tin nhắn tiếp sức trong phạm vi nhóm cục bộ. Yêu cầu tín hiệu xác nhận tương tự tin nhắn cá nhân.
group_sync	group_name, members, creator	Bản tin đồng bộ hóa trạng thái nhóm. Chứa mảng ma trận danh sách các thành viên hiện hành và định danh người khởi tạo nhóm.

Bảng 3.4: Đặc tả thông điệp phát sóng và trò chuyện nhóm

3.6. Thiết kế luồng nghiệp vụ chi tiết

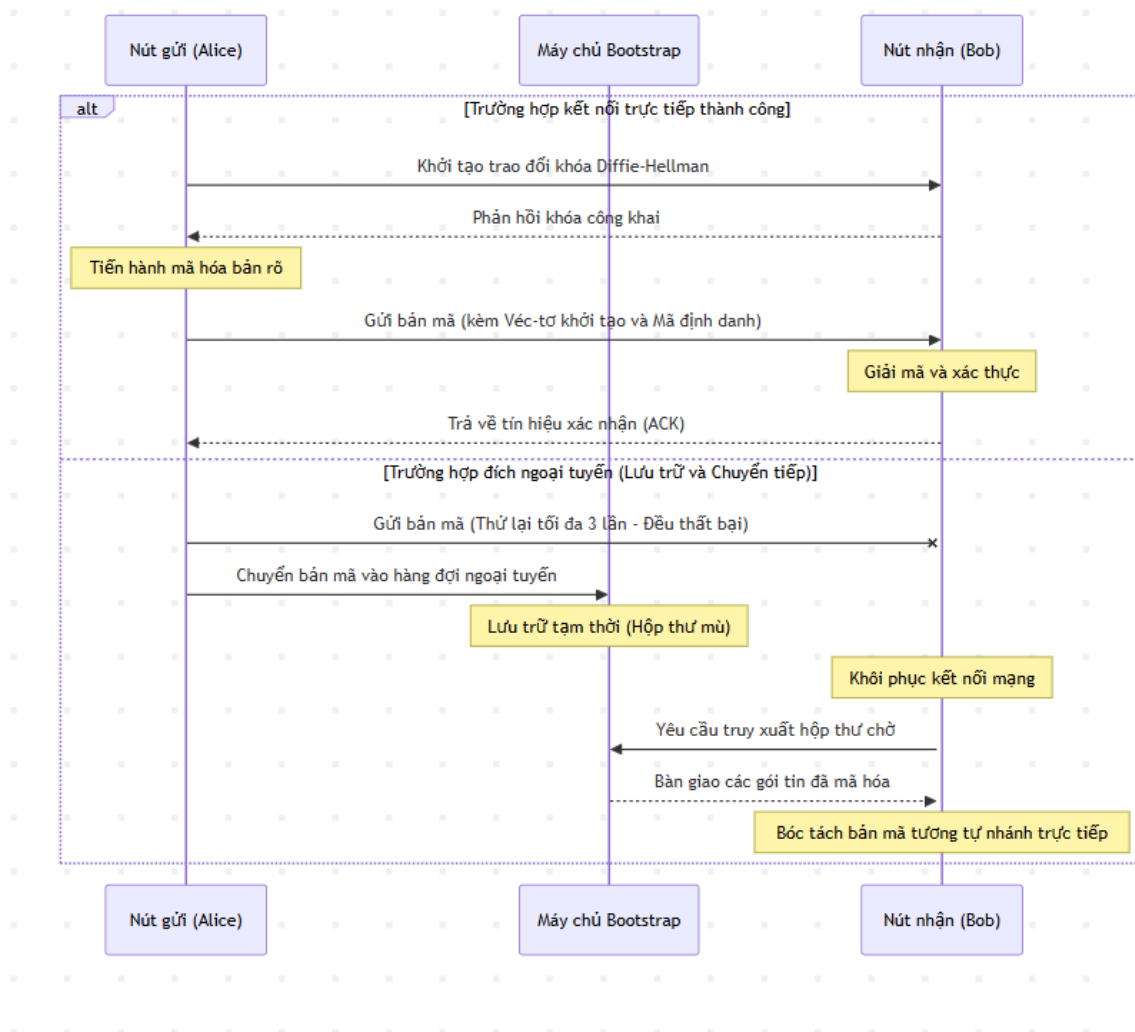
3.6.1. Chu trình gia nhập mạng lưới

Quá trình một nút mạng mới tham gia vào hệ thống là sự phối hợp chặt chẽ giữa tương tác cục bộ và các bản tin điều khiển từ máy chủ. Khởi đầu bằng việc đăng ký định danh, hệ thống tự động kích hoạt các luồng chạy nền, tiến hành đồng bộ hóa trạng thái toàn mạng và xử lý các thông điệp tồn đọng trước khi chính thức cho phép người dùng giao tiếp.



3.6.2. Chu trình truyền tin bảo mật và xử lý ngoại tuyến

Đây là luồng nghiệp vụ cốt lõi chứng minh năng lực của kiến trúc mạng ngang hàng kết hợp. Đối với kịch bản lý tưởng (hai nút cùng trực tuyến), mọi tiến trình từ thỏa thuận khóa bảo mật, bọc mã hóa đến xác nhận nhận (ACK) đều diễn ra hoàn toàn trực tiếp. Trong trường hợp gián đoạn kết nối, cơ chế xử lý ngoại tuyến thông qua hộp thư mù tại máy chủ trung tâm sẽ được kích hoạt để đảm bảo tính toàn vẹn của luồng giao tiếp.



3.6.3. Chu trình phát sóng lan truyền

Để phân phối thông điệp đến toàn bộ mạng lưới mà không phụ thuộc vào một máy chủ phát thanh trung tâm, hệ thống áp dụng thuật toán lan truyền tin đồn với các thông số kiểm soát chặt chẽ.

Khởi đầu, nút nguồn sẽ khởi tạo một mã định danh duy nhất cho thông điệp, cấu hình giới hạn thời gian sống (TTL, mặc định bằng 3 bước nhảy) và đưa mã định danh này vào danh sách bộ nhớ đệm cục bộ. Thông điệp sau đó được phát tán đồng thời tới mọi địa chỉ trong danh sách định tuyến cục bộ thông qua các luồng xử lý song song.

Khi một nút trung gian tiếp nhận thông điệp, quy trình xử lý diễn ra như sau:

1. Kiểm tra trùng lặp: Đối chiếu mã định danh với bộ nhớ đệm. Nếu đã tồn tại, lập tức hủy bỏ gói tin nhằm ngăn chặn tình trạng bão mạng. Nếu chưa tồn tại, hệ thống ghi nhận mã này và tiến hành hiển thị nội dung.
2. Kiểm soát vòng đời: Nếu chỉ số vòng đời vẫn lớn hơn 0, hệ thống tự động giảm giá trị này đi 1 đơn vị.
3. Tiếp tục lan truyền: Nút trung gian đóng vai trò như một trạm phát sóng mới, tiếp tục chuyển tiếp thông điệp đến các nút lân cận, tuyệt đối loại trừ địa chỉ IP của trạm vừa chuyển thông điệp tới để tránh vòng lặp phản hồi ngược. Chu trình lặp lại cho đến khi chỉ số TTL triệt tiêu về 0.

3.6.4. Chu trình quản lý và tiếp sức nhóm phân tán

Việc duy trì một phòng trò chuyện mà không có máy chủ chủ quản đòi hỏi cơ chế quản lý trạng thái phân tán và nguyên lý nhất quán cuối cùng.

- Khởi tạo và kết nạp: Nút khởi tạo tạo ra một vùng nhớ cục bộ chứa cấu trúc nhóm. Khi kết nạp thêm thành viên, nút khởi tạo không chỉ gửi lời mời tham gia đến thành viên mới mà còn phải phát đi một bản tin đồng bộ hóa chứa ma trận danh sách thành viên đầy đủ đến toàn bộ các thành viên hiện hữu trong nhóm.
- Tiếp sức thông điệp: Khi một thành viên phát ngôn, thông điệp được gửi trực tiếp đến toàn bộ danh sách thành viên mà nút đó ghi nhận được. Mỗi thành viên khi nhận được tin nhắn không chỉ hiển thị mà còn thực hiện chức năng tiếp sức, lan truyền chéo gói tin đến các thành viên khác trong nhóm (loại trừ bản thân và nút gửi gốc). Kỹ thuật đa điểm này đảm bảo rằng ngay cả khi có một vài kết nối điểm-điểm bị đứt gãy tạm thời, thông điệp vẫn có thể đi đường vòng qua các thành viên khác để tiếp cận đích, giúp trạng thái thông tin của toàn nhóm hội tụ và đồng nhất.

3.7. Thiết kế xử lý lỗi

Hệ thống phân tán luôn tiềm ẩn nhiều rủi ro về mặt kết nối và toàn vẹn dữ liệu. Nhằm đảm bảo tính bền vững, dự án đã thiết kế và tích hợp một ma trận xử lý ngoại lệ, bao quát các tình huống sự cố tiêu biểu trong quá trình vận hành. Chi tiết các kịch bản và phản ứng của hệ thống được đặc tả trong Bảng 3.5.

Mã rủi ro	Tình huống sự cố	Biện pháp xử lý của hệ thống
ERR-01	Máy chủ Bootstrap không phản hồi(<i>Server unreachable</i>)	Đánh dấu lỗi quy trình đăng ký. Hệ thống tự động thoát tiến trình (exit) đối với giao diện dòng lệnh, hoặc hiển thị hộp thoại cảnh báo đối với giao diện đồ họa.
ERR-02	Xung đột cổng mạng cục bộ(<i>Port bind fail</i>)	Buộc dừng hệ thống khẩn cấp (sys.exit(1)) để giải phóng tài nguyên và tránh các hành vi dị thường.
ERR-03	Thiếu khóa bảo mật chung(<i>Missing shared key</i>)	Từ chối thực thi lệnh bọc mã hóa ở phía gửi và cảnh báo lỗi giải mã ở phía nhận để bảo vệ tính toàn vẹn E2EE.
ERR-04	Trễ hạn xác nhận nhận(<i>ACK timeout</i>)	Kích hoạt chiến lược thử lại. Nếu vượt quá số lần quy định, hệ thống tự động chuyển gói tin lên hàng đợi ngoại tuyến (Store-and-forward).
ERR-05	Cấu trúc JSON không hợp lệ(<i>Invalid JSON format</i>)	Hàm nhận trả về giá trị rỗng (None); hệ thống lập tức đóng phiên kết nối TCP hiện tại để ngăn chặn nguy cơ tiêm nhiễm mã độc hoặc tràn bộ nhớ.
ERR-06	Tệp tin vượt quá 2MB(<i>File size limit exceeded</i>)	Chủ động từ chối xử lý ngay tại vòng kiểm duyệt đầu vào (trước khi nạp vào bộ nhớ chính) để tránh cạn kiệt RAM.
ERR-07	Gói tin phát sóng bị trùng lặp(<i>Broadcast duplication</i>)	Gói tin bị hủy bỏ (drop) ngay lập tức dựa trên việc đối chiếu bộ đệm seen_messages nhằm

		triệt tiêu hiện tượng bão mạng (broadcast storm).
--	--	---

Bảng 3.5: Ma trận các kịch bản xử lý ngoại lệ và dung lỗi

3.8. Thiết kế giao diện

Nhằm phục vụ đa dạng các mục đích từ phát triển, kiểm thử cho đến trải nghiệm người dùng cuối, hệ thống cung cấp hai phương thức giao tiếp độc lập nhưng chia sẻ chung một lõi kiến trúc nghiệp vụ (Data & Control Plane).

Tiêu chí	Giao diện Dòng lệnh (CLI)	Giao diện Đồ họa (GUI - Tkinter)
Mục đích chính	Phục vụ gỡ lỗi (debug), kiểm thử kịch bản xáo trộn mạng (churn) và triển khai trên máy chủ không có giao diện.	Tối ưu hóa trải nghiệm người dùng cuối, theo dõi trực quan luồng sự kiện mạng.
Phương thức điều khiển	Sử dụng tiền tố ! để phân định lệnh hệ thống (Ví dụ: !send, !gcreate). Các luồng không có tiền tố mặc định là lệnh phát sóng.	Điều hướng trực quan qua các biểu mẫu đăng nhập, thanh công cụ (sidebar) và nút bấm tương tác.
Hiển thị luồng dữ liệu	Sử dụng mã màu chuẩn ANSI để phân loại trực tiếp các bản ghi nhật ký (log), lỗi và thông báo hệ thống trên cùng một màn hình Terminal.	Phân mảnh không gian hiển thị: Thanh bên quản lý danh sách thành viên/nhóm; không gian trung tâm mô phỏng bong bóng hội thoại (bubble chat). Lịch sử được tách biệt rõ ràng cho từng luồng.
Cơ chế an toàn luồng(<i>Thread-safety</i>)	In luồng chuẩn ra Terminal (có khóa đồng bộ cơ bản).	Áp dụng mô hình bất đồng bộ: Các luồng mạng đẩy dữ liệu vào <code>gui_queue</code> ; vòng lặp sự kiện chính dùng <code>root.after()</code> quét mỗi 100ms

		để vẽ lại màn hình, tránh xung đột tài nguyên.
--	--	--

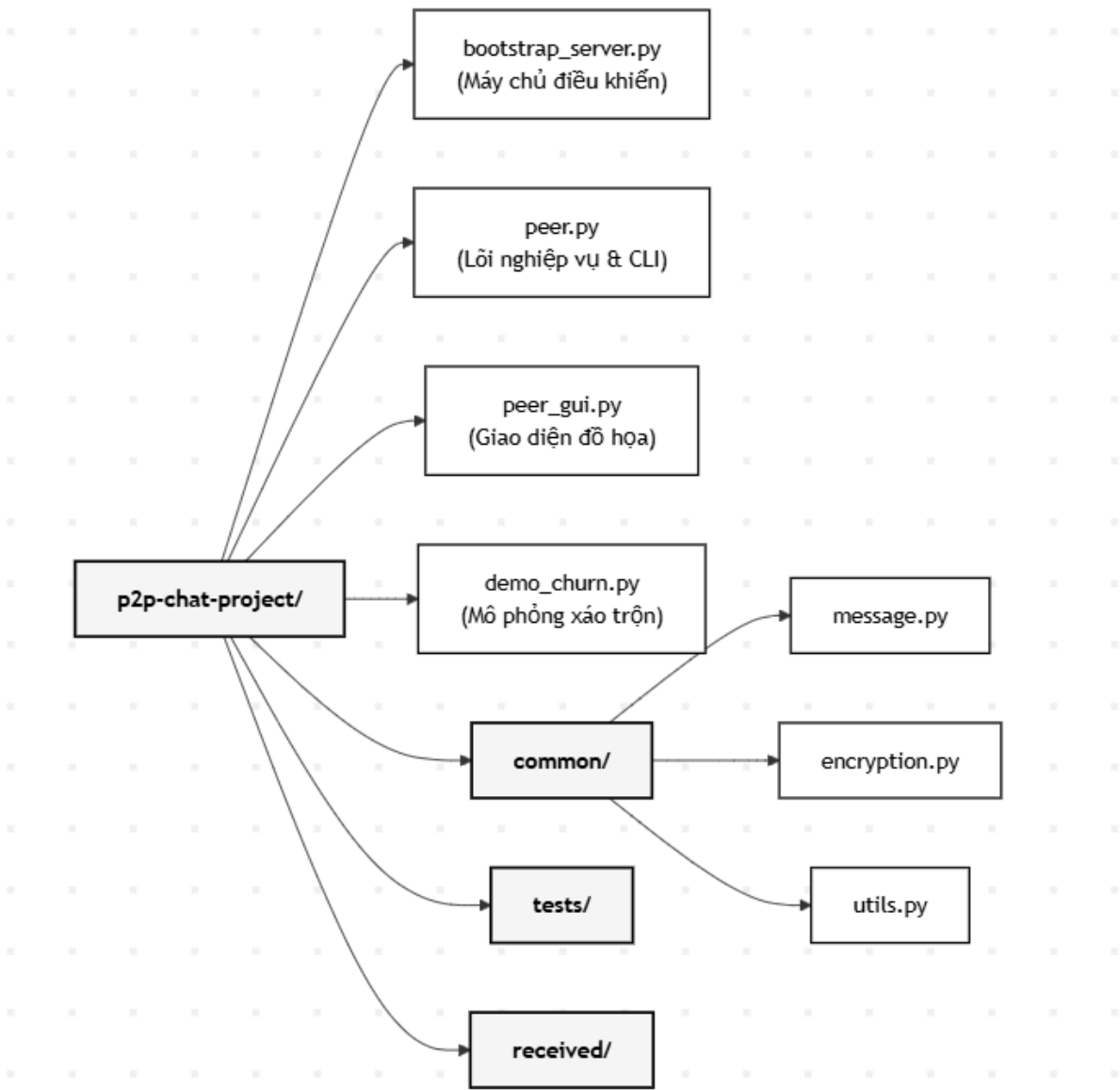
Bảng 3.6: Đặc tả chức năng các môi trường giao diện

Chương 4: Triển khai các cơ chế phân tán

4.1. Tổng quan

4.1.1. Cấu trúc thư mục và phân chia trách nhiệm

Cấu trúc dự án được phân rã thành các tệp thực thi độc lập kết hợp với một gói thư viện dùng chung, nhằm tối ưu hóa khả năng bảo trì và kiểm thử tính năng:



4.1.2. Sự phụ thuộc giữa các mô-đun Kiến trúc phụ thuộc được thiết kế lỏng lẻo. Khối giao diện đồ họa và khối lõi nút mạng chia sẻ chung tập hợp các thư viện tiện ích, bảo mật và cấu trúc thông điệp từ gói common. Trong khi đó, máy chủ điều khiển chỉ kế thừa các tiện ích phân tích chuỗi và truyền tải mạng cơ bản. Điểm đặc biệt của dự án là không phụ thuộc vào bất kỳ thư viện bên thứ ba nào ở môi trường thực thi (ngoại trừ cấu hình kiểm thử), đảm bảo khả năng triển khai linh hoạt trên mọi hệ thống tích hợp sẵn Python.

4.2. Triển khai Bootstrap Server

4.2.1. Khởi tạo và thiết lập thông số

Quá trình khởi động máy chủ tuân thủ trình tự cấp phát tài nguyên mạng tiêu chuẩn. Máy chủ liên tục lắng nghe trên mọi giao diện mạng khả dụng tại cổng mặc định 5555, duy trì hàng đợi tối đa 10 kết nối chờ. Song song với vòng lặp chính, một luồng chạy ngầm được khởi tạo để rà soát và thanh lọc các nút mạng không phát tín hiệu duy trì quá 30 giây, với chu kỳ quét là 5 giây/lần.

Python:

```
Thread(target=self.sweep_inactive_peers, daemon=True).start()
self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR,
1)
self.server_socket.bind(('0.0.0.0', 5555))
self.server_socket.listen(10)
# Vòng lặp chấp nhận kết nối và điều phối sang luồng mới
```

4.2.2. Xử lý yêu cầu kết nối cục bộ

Giao tiếp giữa nút mạng và máy chủ được triển khai theo mô hình đóng phiên độc lập: mỗi kết nối TCP chỉ xử lý duy nhất một khối dữ liệu JSON. Hệ thống tiến hành trích xuất trường định dạng, điều hướng dữ liệu đến các hàm nghiệp vụ tương ứng và chủ động đóng luồng để thu hồi tài nguyên

Python:

```
msg = recv_json(conn, timeout=5)
if not msg:
    return
msg_type = msg.get("type")
# Điều phối luồng xử lý theo msg_type
```

```
#finally: conn.close()
```

4.2.3. Triển khai nghiệp vụ ghi danh mạng

Việc cập nhật sổ đăng ký được thực thi tuần tự thông qua cơ chế khóa đồng bộ nhằm đảm bảo an toàn vùng nhớ. Hệ thống áp dụng thuật toán tìm kiếm tuyến tính để kiểm tra sự tồn tại của định danh trước khi ghi đè hoặc bổ sung bản ghi mới. Điểm nhấn trong triển khai là lệnh kích hoạt thông báo hiện diện (`_notify_all_peers`) được cố tình đặt ngoài phạm vi khóa đồng bộ nhằm triệt tiêu hoàn toàn rủi ro bế tắc tài nguyên (deadlock).

4.2.4. Triển khai hàng đợi thông điệp ngoại tuyến

Hộp thư ngoại tuyến được tổ chức dưới dạng cấu trúc từ điển, sử dụng cặp địa chỉ định tuyến làm khóa tra cứu. Trong phạm vi dự án, hàng đợi này không bị giới hạn về dung lượng hay vòng đời lưu trữ.

Python:

```
# Tiến trình lưu trữ chờ
```

```
key = (target_ip, target_port)
```

```
with self.lock:
```

```
    self.pending_messages.setdefault(key, []).append(payload)
```

```
# Tiến trình giải phóng hàng đợi
```

```
with self.lock:
```

```
    pending = self.pending_messages.pop(key, [])
```

```
return {"type": "pending_batch", "messages": pending}
```

Khi phát sinh lệnh truy xuất, lệnh pop được sử dụng để bàn giao và xóa sạch hàng đợi ngay lập tức. Cơ chế này áp đặt ngữ nghĩa phân phối "nhiều nhất một lần" từ phía máy chủ; nếu nút nhận xảy ra sự cố đột ngột trong lúc tải dữ liệu, thông điệp có nguy cơ bị thất lạc.

4.2.5. Phân phối thông báo chủ động

Các luồng đẩy sự kiện mạng được cấu hình với giới hạn thời gian chờ cực ngắn (3 giây) nhằm không làm gián đoạn luồng quét dọn tổng thể.

Python:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

```
s.settimeout(3)
s.connect((ip, int(port)))
send_json(s, msg_dict)
```

Giao thức đẩy thông báo tuân thủ triết lý "nỗ lực tối đa" (best-effort). Mọi ngoại lệ xảy ra trong quá trình thiết lập kết nối đều bị bỏ qua một cách có chủ đích (except: pass), bảo vệ tiến trình của máy chủ khỏi các nút mạng dị thường.

4.3. Triển khai Peer - khởi động và control plane

4.3.1. Điểm neo khởi động giao diện dòng lệnh

Hệ thống tiếp nhận cấu hình vận hành thông qua các tham số đầu vào khi khởi chạy tệp tin peer.py. Bảng 4.1 trình bày chi tiết tập lệnh cấu hình cơ sở và các tham số phục vụ kiểm thử xáo trộn mạng.

Tham số cấu hình	Giá trị mặc định	Diễn giải nghiệp vụ
--ip / --port	127.0.0.1 / (Yêu cầu nhập)	Địa chỉ và cổng mạng cục bộ để cấp phát máy chủ lắng nghe.
--name	(Yêu cầu nhập)	Định danh hiển thị của người dùng trên toàn mạng.
--bootstrap-host / -port	127.0.0.1 : 5555	Địa chỉ định tuyến của máy chủ điều khiển mạng lưới.
--churn	false	Kích hoạt kịch bản mô phỏng ra/vào mạng liên tục.
--churn-online-seconds	20	Thời gian duy trì trạng thái trực tuyến trong một chu kỳ mô phỏng.
--churn-offline-seconds	10	Thời gian giả lập mất kết nối mạng.
--churn-cycles	0 (Vô hạn)	Số lượng chu kỳ thực thi mô phỏng.

Bảng 4.1: Tham số cấu hình khởi động nút mạng

Sau khi nạp cấu hình và ghi danh thành công với máy chủ, hệ thống chuyển cờ trạng thái sang trực tuyến và tuần tự kích hoạt các luồng xử lý nền trước khi nhường quyền điều khiển cho vòng lặp giao diện chính:

Python:

```
peer.online = True
Thread(target=peer.start_server, daemon=True).start()
Thread(target=peer.heartbeat_loop, daemon=True).start()
Thread(target=peer.update_peer_list_loop, daemon=True).start()
peer.fetch_pending_messages() # Truy xuất hộp thư ngay lập tức
```

if args.churn:

```
    Thread(target=peer.churn_loop, ...).start()
```

```
peer.cli_loop() # Vòng lặp khóa luồng chính (blocking main)
```

4.3.2. Triển khai thủ tục ghi danh

Khởi logic đăng ký thiết lập một kết nối ngắn hạn tới máy chủ trung tâm. Hệ thống áp đặt giới hạn thời gian chờ 5 giây để ngăn chặn tình trạng treo luồng cục bộ. Khởi lệnh `finally` đảm bảo tài nguyên kết nối luôn được thu hồi độc lập với kết quả giao dịch. Nếu kết nối thất bại, hệ thống lập tức xuất chuỗi cảnh báo định dạng màu đỏ ra thiết bị xuất chuẩn.

Python:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.settimeout(5)
try:
    s.connect(bootstrap_addr)
    send_json(s, register_msg)
    res = recv_json(s, timeout=5)
    return res.get("status") == "success"
finally:
    s.close()
```

4.3.3. Đồng bộ danh sách mạng và hộp thư

Luồng chạy ngầm này duy trì chu kỳ nghỉ 10 giây giữa các lần thực thi (với khoảng trễ khởi tạo 2 giây để chờ máy chủ ổn định). Nhiệm vụ của luồng là đảm bảo tính cập nhật của danh sách định tuyến cục bộ, đồng thời liên tục thăm dò và giải phóng các thông điệp tồn đọng tại máy chủ.

Python:

```
time.sleep(2)
while running:
    if online:
        get_peer_list()
        fetch_pending_messages()
        time.sleep(10)
```

4.3.4. Luồng duy trì nhịp tim

Luồng nhịp tim báo cáo trạng thái định kỳ 15 giây một lần. Khác với quy trình ghi danh, lệnh gửi nhịp tim tuân thủ triết lý "nỗ lực tối đa", không kiểm tra phản hồi từ máy chủ nhằm giảm thiểu tài nguyên xử lý. Bản thân nút mạng không tự đánh giá trạng thái mất kết nối nếu nhịp tim gửi lỗi, mà phụ thuộc hoàn toàn vào cơ chế quét và đẩy thông báo từ máy chủ trung tâm.

Python:

```
while running:
    time.sleep(15)
    if running and online:
        send_heartbeat()
```

4.4. Triển khai TCP Server phía Peer

4.4.1. Khởi tạo cổng lắng nghe

Mỗi nút mạng sở hữu một luồng máy chủ cục bộ độc lập. Hệ thống sử dụng cờ cấu hình SO_REUSEADDR để cho phép tái sử dụng cổng mạng ngay lập tức trong các tình huống khởi động lại đột ngột, tránh ngoại lệ xung đột địa chỉ hệ thống. Nếu quy trình cấp phát cổng thất bại, tiến trình sẽ báo lỗi và chủ động gọi sys.exit(1) để kết thúc.

Python:

```
server_socket.bind((self.ip, self.port))
server_socket.listen(10)
while self.running:
    conn, addr = server_socket.accept()
    Thread(target=self.handle_incoming, args=(conn, addr),
daemon=True).start()
```

4.4.2. Xử lý phiên giao tiếp đầu vào

Nguyên lý cơ bản của mặt phẳng dữ liệu là mỗi phiên kết nối thông thường chỉ giải quyết duy nhất một thông điệp (ngoại trừ thao tác gửi tín hiệu xác nhận được thực hiện ngay trên cùng kết nối đó trước khi đóng). Cơ chế vòng đời ngắn này được bảo vệ bởi giới hạn thời gian chờ đọc 10 giây, giúp hệ thống kháng cự lại các tình trạng treo kết nối vô hạn.

Python:

```
def handle_incoming(self, conn, addr):
    try:
        msg = recv_json(conn, timeout=10)
        if not msg:
            return
        self._process_incoming_message(conn, addr, msg)
    except:
        pass
    finally:
        conn.close()
```

4.5. Triển khai E2EE

4.5.1. Thuật toán trao đổi khóa Diffie-Hellman (common/encryption.py)

Hệ thống sử dụng số nguyên tố 512-bit chuẩn RFC để thực hiện trao đổi khóa. Khóa bí mật chung sau khi sinh ra sẽ được băm qua hàm SHA-256 để tạo thành khóa đối xứng.

Python:

```
DH_PRIME = 0xFD7F5381... # Số nguyên tố 512-bit
DH_GENERATOR = 2
private_key = int.from_bytes(os.urandom(32), 'big')
public_key = pow(DH_GENERATOR, private_key, DH_PRIME)
```

```
def generate_shared_key(self, peer_public_key):
    shared_secret = pow(peer_public_key, self.private_key, DH_PRIME)
    return hashlib.sha256(str(shared_secret).encode('utf-8')).digest()
```

Lưu ý triển khai: Việc ép kiểu khóa bí mật sang chuỗi văn bản (str) trước khi băm là giải pháp đơn giản phục vụ môi trường mô phỏng. Trên thực tế, hệ thống cấp doanh nghiệp sẽ sử dụng mảng byte có độ dài cố định.

4.5.2. Mã hóa luồng đối xứng (Stream Cipher)

Dữ liệu được mã hóa và giải mã thông qua phép toán XOR giữa bản rõ và chuỗi khóa (Keystream). Chuỗi khóa này được sinh liên tục bằng hàm SHA-256 kết hợp với Véc-tơ khởi tạo (IV) và biến đếm.

Python:

```
def encrypt_decrypt_bytes(data, key, iv):
    counter = 0
    for i in range(0, len(data), 32):
        block = iv + counter.to_bytes(4, 'big')
        keystream = hashlib.sha256(key + block).digest()
        # Áp dụng phép toán XOR từng byte
        counter += 1
```

Thành phần	Kích thước	Ghi chú
Véc-tơ khởi tạo (IV)	16 bytes	Sinh ngẫu nhiên cho mỗi gói tin.
Biến đếm (Counter)	4 bytes	Chuẩn Big-endian, chống lặp chuỗi khóa.

Khối khóa (Keystream)	32 bytes	Phù hợp với đầu ra của hàm băm SHA-256.
--------------------------	----------	--

Đối với văn bản, hệ thống sử dụng `encrypt_string` / `decrypt_string` để mã hóa UTF-8 và chuyển đổi sang chuẩn Base64 nhằm truyền tải an toàn qua định dạng JSON.

4.5.3. Khởi tạo tiến trình thỏa thuận khóa (`ensure_shared_key`)

Hàm này chịu trách nhiệm thiết lập kênh bảo mật. Nó ưu tiên truy xuất khóa từ bộ nhớ đệm; nếu chưa tồn tại, hệ thống chủ động mở kết nối TCP để gửi bản tin bắt tay.

Python:

with self.lock:

if (target_ip, target_port) in self.shared_keys:
return cached_key

dh = DiffieHellman()

handshake_msg = {"type": "key_exchange", "pub_key": dh.public_key, ...}

s.connect((target_ip, target_port))

send_json(s, handshake_msg)

res = recv_json(s, timeout=5)

if res.get("type") == "key_exchange_reply":

key = dh.generate_shared_key(res.get("pub_key"))

self.shared_keys[(target_ip, target_port)] = key

4.5.4. Xử lý phản hồi trao đổi khóa (`_handle_key_exchange`)

Tại nút nhận, hệ thống tính toán khóa bí mật và lập tức gửi trả khóa công khai của mình ngay trên cùng một phiên kết nối TCP đang mở.

Python:

dh_b = DiffieHellman()

shared_key = dh_b.generate_shared_key(peer_pub_key)

self.shared_keys[(from_ip, from_port)] = shared_key

```
# Phản hồi trực tiếp trên kết nối hiện tại
send_json(conn, {"type": "key_exchange_reply", "pub_key": dh_b.public_key})
```

4.5.5. Luồng gửi và nhận thông điệp cá nhân

Tiến trình truyền tải kết hợp chặt chẽ giữa tính năng bảo mật đầu cuối và cơ chế kiểm soát lỗi ở tầng ứng dụng:

- Khối gửi: Bọc mã hóa thông điệp, đính kèm mã định danh và đợi tín hiệu xác nhận (ACK). Nếu mạng gặp sự cố, đẩy bản mã vào hộp thư ngoại tuyến.
- Khối nhận: Giải mã bản mã, gửi phản hồi xác nhận và hiển thị nội dung.

Python:

```
# Tiết diện luồng gửi
target_ip, target_port, name = _resolve_peer(target)
shared_key = ensure_shared_key(...)
b64_cipher, b64_iv = encrypt_string(content, shared_key)
chat_msg = {"type": "chat", "msg_id": str(uuid.uuid4()), "ciphertext":
b64_cipher, "iv": b64_iv, ...}

if not send_with_ack(target_ip, target_port, chat_msg):
    _store_offline_message(...)

# Tiết diện luồng nhận
shared_key = self.shared_keys.get((sender_ip, from_port))
content = decrypt_string(ciphertext, shared_key, iv)

if conn:
    _send_ack(conn, msg_id) # Không phản hồi ACK đối với các tin nhắn chờ
    (pending)
print("[I-I Chat] ...")
```

4.6. Triển khai Reliable Delivery

4.6.1. Chiến lược thử lại với độ trễ tăng lũy thừa (send_reliable)

Để đối phó với sự cố rớt mạng chập chờn, hệ thống tích hợp thuật toán tăng trễ lũy thừa (Exponential Backoff). Nếu việc gửi tin thất bại, hệ thống không thử lại liên tục mà sẽ tăng dần thời gian chờ sau mỗi lần thử.

Python:

```
for attempt in range(max_retries): # Mặc định 3 lần
    s.connect((target_ip, target_port))
    send_json(s, msg_dict)
    ack = recv_json(s, timeout=timeout)

    # Kiểm tra tính hợp lệ của tín hiệu xác nhận
    if ack.get("type") == "ack" and ack.get("msg_id") ==
msg_dict.get("msg_id"):
        return True, ack

    backoff = min(2 ** attempt, 8)
    time.sleep(backoff)
return False, None
```

Lần thử thất bại	Độ trễ áp dụng (Backoff)	Đánh giá Worst-case (Tình huống xấu nhất)
Lần 0	1 giây	Tổng thời gian trễ trước khi hệ thống đánh giá đường truyền đứt gãy hoàn toàn: $\approx 3 \times (5s \text{ timeout}) + (1 + 2 + 4) = 22 \text{ giây}$
Lần 1	2 giây	
Lần 2	4 giây	

Bảng 4.3: Tham số và thời gian độ trễ lũy thừa

4.6.2. Phân nhánh xử lý sau khi thất bại toàn tập

Khi hàm `send_reliable` trả về trạng thái lỗi (vượt ngưỡng 22 giây), hệ thống sẽ từ bỏ việc truyền tải điểm-điểm và chuyển hướng lưu trữ gói tin lên máy chủ trung tâm (`_store_offline_message`). Lệnh gửi lên máy chủ được thiết kế là lệnh gửi đơn (không áp dụng thử lại) để tránh tắc nghẽn luồng.

4.7. Triển khai Broadcast Epidemic

4.7.1. Khởi tạo và Phân phối song song

Khi người dùng phát sóng (hoặc gõ văn bản không có tiền tố ! trên giao diện dòng lệnh), hệ thống sinh ra một mã định danh duy nhất (UUID), tự động thêm mã này vào bộ nhớ đệm `seen_messages` để tránh tự phản hồi. Thông điệp được phân phối tới tất cả các nút lân cận thông qua cơ chế đa luồng song song (Parallel fan-out) nhằm giảm thiểu độ trễ mạng.

Python:

```
msg_id = str(uuid.uuid4())
broadcast_msg = {"type": "broadcast", "content": content, "msg_id": msg_id,
                "ttl": 3, ...}
```

with lock:

```
seen_messages.add(msg_id)
```

for p in active_peers:

```
Thread(target=send_direct_message, args=(p.ip, p.port, broadcast_msg),
daemon=True).start()
```

4.7.2. Xử lý lan truyền tại nút trung gian

Khi tiếp nhận gói tin phát sóng, nút mạng sẽ đối chiếu mã định danh để loại trừ gói tin lặp. Nếu hợp lệ, hệ thống tiến hành giảm chỉ số vòng đời (ttl) và tiếp tục chuyển tiếp thông điệp.

Python:

with lock:

```

if msg_id in seen_messages:
    return
seen_messages.add(msg_id)

if ttl > 0:
    forward_msg = {..., "ttl": ttl - 1}
    for p in copy(peer_list):
        if p.ip == addr[0]:
            continue # Heuristic: Bỏ qua IP của chính nút vừa gửi tới
        Thread(target=send_direct_message, ...).start()

```

Lưu ý giới hạn: Thuật toán tối ưu chặn vòng lặp dựa trên địa chỉ IP (addr[0]). Mặc dù hoạt động hoàn hảo trên môi trường mạng cục bộ, nhưng khi kiểm thử mô phỏng nhiều nút trên cùng một máy (localhost 127.0.0.1), cơ chế này có thể chặn nhầm các cổng khác nhau.

4.8. Triển khai Group Chat

4.8.1. Kiểm soát cấu trúc trạng thái nhóm

Trạng thái nhóm được duy trì độc lập tại bộ nhớ RAM của từng nút mạng. Khi một nút mạng khởi động lại, toàn bộ thông tin cục bộ về nhóm sẽ bị xóa bỏ.

Python:

```

self.groups["team_name"] = {
    "members": [("127.0.0.1", 5001, "Alice"), ("127.0.0.1", 5002, "Bob")],
    "creator": "Alice"
}

```

4.8.2. Khởi tạo và Đồng bộ hóa thành viên

Quá trình tạo nhóm ban đầu chỉ diễn ra cục bộ. Chỉ khi có lệnh thêm thành viên (handle_group_add), hệ thống mới bắt đầu quá trình đồng bộ hóa qua mạng. Nút quản trị sẽ gửi bản tin group_join tới thành viên mới, đồng thời gửi bản tin group_sync (chứa toàn bộ ma trận thành viên) đến tất cả các thành viên hiện hữu nhằm đạt được sự nhất quán cuối cùng.

Python:

```
join_msg = {"type": "group_join", ...}
for m in all_members:
    if m is self: continue
    send_direct_message(m_ip, m_port, join_msg)
    _sync_group_to_member(m_ip, m_port, group_name) # Gửi cấu trúc mảng
members
```

4.8.3. Tiếp sức thông điệp đa điểm (_handle_group_msg)

Khi một thông điệp nhóm được gửi đi, nó được thiết kế để tự động tiếp sức qua các thành viên. Khởi nhận sẽ kiểm tra tính trùng lặp, phản hồi tín hiệu xác nhận (ACK) về cho người gửi, sau đó chuyển tiếp bản sao thông điệp đến các thành viên còn lại (loại trừ bản thân và địa chỉ nút vừa gửi).

Python:

```
if msg_id in seen_messages:
    return
seen_messages.add(msg_id)
_send_ack(conn, msg_id)

for m in members:
    if m is self or m.ip == addr[0]:
        continue
    Thread(target=send_direct_message, args=(..., forward_msg)).start()
```

Mâu thuẫn thiết kế có chủ đích: Nút gửi ban đầu không kích hoạt chờ đợi tín hiệu xác nhận đồng bộ; tuy nhiên, các nút nhận tiếp sức lại chủ động gửi trả tín hiệu ACK nhằm tăng cường độ tin cậy của luồng dữ liệu cục bộ.

4.8.4. Xử lý sự kiện rời nhóm (handle_group_leave_cmd)

Khi người dùng chủ động rời nhóm, hệ thống xóa bỏ dữ liệu cấu trúc nhóm trên bộ nhớ cục bộ, sau đó phát đi thông báo rời nhóm. Các nút khác khi nhận thông báo này (_handle_group_leave) chỉ tiến hành loại bỏ định danh của người rời đi

khởi mảng `members`, hoàn toàn không xóa bỏ cấu trúc nhóm gốc trừ khi không còn thành viên nào.

4.9. Triển khai Store-and-Forward & Churn

4.9.1. Luồng xử lý thông điệp ngoại tuyến (Store-and-Forward)

Khi quá trình truyền tải trực tiếp thất bại do nút nhận mất kết nối, hệ thống chủ động đẩy bản mã lên máy chủ trung tâm. Khi khôi phục kết nối, nút nhận tự động truy xuất hộp thư chờ và tiến hành giải mã cục bộ mà không phát sinh tín hiệu xác nhận ngược.

Python:

```
# Kịch bản nút gửi (Alice) xử lý khi đích (Bob) ngoại tuyến
bob.server_socket.close()
bob.running = False
alice.send_with_ack(..., max_retries=1) # Trả về False
alice._store_offline_message(bob.ip, bob.port, offline_msg) # Chuyển tiếp
thành công (True)

# Kịch bản khôi phục tại nút nhận (Bob)
bob.start_server()
bob.register_with_bootstrap()
bob.fetch_pending_messages()
# Khởi chạy luồng: _process_incoming_message(conn=None, ...)
# Giải mã bản rõ và bỏ qua bước gửi ACK.
```

4.9.2. Kịch bản mô phỏng xáo trộn mạng (churn_loop)

Để kiểm thử độ bền bỉ của hệ thống, luồng mô phỏng được lập trình để liên tục thay đổi trạng thái trực tuyến/ngoại tuyến của nút mạng theo chu kỳ định trước.

Python:

```
while running and (cycles is None or cycle_count < cycles):
    sleep(online_seconds)
    online = False
    leave_network()
```

```

sleep(offline_seconds)
if register_with_bootstrap():
    online = True
    get_peer_list()
    fetch_pending_messages()
    cycle_count += 1

```

4.9.3. Kịch bản kiểm thử toàn trình (demo_churn.py)

Dự án cung cấp một kịch bản kiểm thử tự động, khởi tạo cùng lúc máy chủ điều khiển và ba nút mạng độc lập. Kịch bản mô phỏng luồng giao tiếp cá nhân, truyền tệp tin song song với hiện tượng rớt mạng có chủ đích để xác thực khả năng phục hồi dữ liệu từ hàng đợi ngoại tuyến.

4.10. Triển khai chuyển file

4.10.1. Luồng kiểm duyệt và gửi tệp (handle_sendfile_command)

Trình tự	Tác vụ kỹ thuật	Hành vi triển khai
1	Xác thực tồn tại	Sử dụng <code>os.path.exists</code> để kiểm tra tính hợp lệ của đường dẫn.
2	Kiểm soát kích thước	Từ chối ngay lập tức nếu dung lượng vượt ngưỡng an toàn ($2 \times 1024 \times 1024$ bytes).
3	Nạp dữ liệu	Đọc toàn bộ nội dung dưới dạng nhị phân (rb).
4	Bọc mã hóa	Gọi hàm <code>encrypt_bytes</code> , sau đó chuyển đổi sang chuỗi Base64 để tương thích chuẩn JSON.
5	Định tuyến	Gửi kèm tín hiệu xác nhận (ACK). Khởi động luồng ngoại tuyến nếu thất bại.

Bảng 4.4: Các bước kiểm duyệt và đóng gói tệp tin

4.10.2. Luồng tiếp nhận và giải mã (_handle_file)

Để ngăn chặn lỗ hổng bảo mật tấn công vượt thư mục (Path Traversal), hệ thống sử dụng hàm `os.path.basename` nhằm vô hiệu hóa các ký tự điều hướng bất hợp pháp trước khi ghi dữ liệu xuống đĩa cứng cục bộ.

Python:

```
ciphertext = base64.b64decode(b64_ciphertext)
iv = base64.b64decode(b64_iv)
file_data = decrypt_bytes(ciphertext, shared_key, iv)

os.makedirs("received", exist_ok=True)
safe_filename = os.path.basename(filename) # Chống kỹ thuật ../../../
dest_path = os.path.join("received", safe_filename)

with open(dest_path, "wb") as f:
    f.write(file_data)
_send_ack(conn, msg_id)
```

4.11. Triển khai GUI (peer_gui.py)

4.11.1. Khác biệt kiến trúc so với Giao diện dòng lệnh

Thành phần kiến trúc	Giao diện Dòng lệnh (CLI)	Giao diện Đồ họa (GUI)
Cơ chế thu thập đầu vào	Hàm chặn luồng <code>input(">> ")</code> .	Ô nhập liệu (Entry) kết hợp điều hướng sự kiện nút bấm.
Cơ chế xuất dữ liệu	Ghi log trực tiếp lên luồng chuẩn.	Đẩy dữ liệu vào hàng đợi cục bộ, vẽ bong bóng hội thoại.
Quản lý không gian nhớ	Không lưu vết hội thoại.	Phân lập lịch sử vào ba vùng nhớ độc lập (<code>peer_chats</code> , <code>group_chats</code> , <code>broadcast_history</code>).

Lỗi logic mạng	Thực thi trực tiếp trên đối tượng Peer.	Tái sử dụng hoàn toàn lỗi mạng, chỉ ghi đè phương thức hiển thị.
----------------	---	--

Bảng 4.5: Đối chiếu đặc điểm triển khai giao diện

4.11.2. Mẫu thiết kế an toàn luồng (Thread-safe UI)

Khung phần mềm Tkinter nghiêm cấm các luồng chạy ngầm can thiệp trực tiếp vào giao diện hiển thị. Dự án xử lý triệt để ràng buộc này bằng mô hình thiết kế bất đồng bộ thông qua hàng đợi tuần tự.

Python:

```
# Luồng mạng chạy ngầm: Đẩy sự kiện thay vì gọi hàm giao diện
self.gui_queue.put(("chat", peer_name, content, is_outgoing))
```

```
# Luồng giao diện chính: Quét hàng đợi mỗi 100ms
def _process_queue(self):
    while not self.gui_queue.empty():
        event = self.gui_queue.get()
        # Xử lý kết xuất (render) bong bóng hội thoại hoặc cập nhật danh sách
        self.root.after(100, self._process_queue)
```

4.11.3. Trình tự khởi tạo giao diện (_handle_join)

Quy trình nạp giao diện tuân thủ nghiêm ngặt nguyên lý kết nối mạng thành công trước khi cấp phát vùng nhìn (view) không gian hội thoại chính.

Python:

```
def _handle_join(self):
    if not self._register_with_bootstrap():
        # Hiển thị cảnh báo lỗi nếu máy chủ từ chối
        return

    Thread(target=self._start_server, daemon=True).start()
    Thread(target=self._heartbeat_loop, daemon=True).start()
```

```

self.login_frame.destroy()
self._build_main() # Kết xuất không gian hội thoại chính

Thread(target=self._get_peer_list, daemon=True).start()
Thread(target=self._fetch_pending_messages, daemon=True).start()

```

4.12. Triển khai đồng bộ và xử lý lỗi

4.12.1. Quản lý phân mảnh bộ nhớ dùng chung

Để triệt tiêu hiện tượng tương tranh (Race condition) trong môi trường đa luồng, toàn bộ các vùng nhớ phân mảnh đều được bảo vệ bởi cơ chế khóa đồng bộ cấp độ đối tượng (self.lock).

Vùng tài nguyên phân mảnh	Đối tượng quản lý	Mục đích bảo vệ
peer_list	Nút mạng cục bộ	Đảm bảo tính nhất quán của danh sách định tuyến.
shared_keys	Nút mạng cục bộ	Bảo vệ tính toàn vẹn của tiến trình tạo khóa Diffie-Hellman.
seen_messages	Nút mạng cục bộ	Tránh lỗi xung đột khi nhiều luồng phát sóng truyền tới cùng lúc.
groups	Nút mạng cục bộ	Bảo vệ ma trận thành viên trong tiến trình đồng bộ nhóm.
peers / pending	Máy chủ Bootstrap	Ngăn ngừa tham chiếu rỗng khi quét dọn các nút bị rút mạng.

Bảng 4.6: Các vùng nhớ yêu cầu cấp phép khóa đồng bộ

4.12.2. Trình phân giải định danh linh hoạt (_resolve_peer)

Hệ thống cho phép người dùng giao tiếp thông qua cả hai hình thức: chỉ số tự sinh hoặc địa chỉ định tuyến trực tiếp, gia tăng tính linh hoạt khi thao tác trên dòng lệnh.

Python:

```
def _resolve_peer(self, target):  
    try:  
        idx = int(target) - 1  
        return self.peer_list[idx] # Truy xuất dựa trên chỉ số giao diện (1-based  
index)  
    except ValueError:  
        if ":" in target:  
            ip, port = target.split(":")  
            return ip, int(port), "Unknown" # Định tuyến ngoại tuyến trực tiếp
```

4.12.3. Bảng tổng hợp rủi ro thường gặp

Dấu hiệu nhận biết	Căn nguyên kỹ thuật	Phương thức can thiệp của hệ thống
Đăng ký thất bại	Máy chủ điều khiển chưa khởi chạy hoặc sai cổng kết nối.	Thoát luồng thực thi an toàn, bung cảnh báo lỗi.
Từ chối giải mã E2EE	Thiếu gói tin bắt tay (Handshake) khởi tạo.	Xuất cảnh báo bảo mật, không làm gián đoạn chương trình chính.
Mất tín hiệu xác nhận	Nút nhận rớt mạng hoặc có tường lửa ngăn chặn.	Kích hoạt luồng trễ lũy thừa; chuyển vị lên máy chủ lưu trữ.
Lỗi Address in use	Khởi động lại khi hệ điều hành chưa giải phóng cổng.	Gửi tín hiệu dừng hệ thống chủ động.
Phân giải JSON lỗi	Dữ liệu bị cắt xén (Corrupt data) trên đường truyền.	Trả về kết quả rỗng, hủy bỏ phiên kết nối lập tức.

Chương 5: Cài đặt và Demo

5.1. Môi trường triển khai

5.1.1. Yêu cầu phần cứng và phần mềm

Để đảm bảo hệ thống vận hành ổn định, môi trường triển khai cần đáp ứng các tiêu chuẩn cấu hình cơ bản được trình bày trong Bảng 5.1.

Thành phần	Tiêu chuẩn cấu hình
Hệ điều hành	Khả năng đa nền tảng: Windows 10/11, Linux, hoặc macOS.
Môi trường Python	Phiên bản 3.8 trở lên (Khuyến nghị sử dụng phiên bản 3.10+).
Hạ tầng mạng	Mạng cục bộ (LAN) cùng dải IP (subnet) hoặc máy trạm cục bộ (localhost).
Bộ nhớ trong (RAM)	Cấp phát khoảng 256MB cho mỗi nút mạng và máy chủ điều khiển.
Dung lượng ổ cứng	Khoảng 50MB (Bao gồm không gian lưu trữ mã nguồn và thư mục nhận tệp tin).

Bảng 5.1: Yêu cầu cấu hình hệ thống tối thiểu

5.1.2. Cài đặt thư viện phụ thuộc

- Môi trường thực thi: Dự án được thiết kế để vận hành hoàn toàn độc lập dựa trên các thư viện tiêu chuẩn của Python (bao gồm socket, threading, json, tkinter...). Do đó, quá trình vận hành thực tế không yêu cầu cài đặt thêm bất kỳ thư viện bên thứ ba nào.
- Môi trường phát triển và kiểm thử: Để phục vụ quá trình chạy các kịch bản kiểm tra tự động, bộ thư viện pytest được đóng gói kèm theo mã nguồn. Việc cài đặt được thực hiện qua giao diện dòng lệnh:

```
cd p2p-chat-project
py -m pip install -r requirements-dev.txt
```

5.1.3. Cấu trúc thư mục vận hành

Khi triển khai ứng dụng, cây thư mục hệ thống được phân bổ như sau:

```
p2p-chat-project/
├── bootstrap_server.py    # Máy chủ điều khiển (Yêu cầu khởi chạy đầu tiên)
├── peer.py                # Nút mạng (Chạy trên giao diện dòng lệnh)
├── peer_gui.py            # Nút mạng (Chạy trên giao diện đồ họa)
├── demo_churn.py          # Kịch bản tự động mô phỏng tích hợp
├── received/              # Thư mục lưu trữ tệp (Hệ thống tự tạo khi nhận file)
└── tests/                 # Tập hợp các kịch bản kiểm thử (pytest)
```

5.2. Hướng dẫn triển khai và khởi động

5.3.1. Kịch bản khởi động tiêu chuẩn (Môi trường cục bộ)

Để thiết lập một mạng lưới tối thiểu, hệ thống yêu cầu khởi chạy một máy chủ điều khiển trung tâm và ít nhất hai nút mạng độc lập.

- Bước 1: Khởi động Máy chủ Điều khiển (Cửa sổ dòng lệnh 1) Di chuyển đến thư mục chứa mã nguồn và thực thi tệp tin khởi chạy máy chủ:

```
cd e:\Downloads\PhanTanFinal\p2p-chat-project
py bootstrap_server.py
```

Kết quả phản hồi mong đợi trên màn hình: [Bootstrap] Server running on 0.0.0.0:5555

- Bước 2: Khởi động Nút mạng Alice (Cửa sổ dòng lệnh 2) Khởi tạo thành viên đầu tiên của mạng lưới bằng giao diện dòng lệnh:

```
py peer.py --name Alice --port 5001
```

- Bước 3: Khởi động Nút mạng Bob (Cửa sổ dòng lệnh 3) Khởi tạo thành viên thứ hai để thiết lập giao tiếp ngang hàng:

```
py peer.py --name Bob --port 5002
```

- Bước 4: Khởi động Nút mạng Carol (Tùy chọn, Cửa sổ dòng lệnh 4) Thêm thành viên thứ ba để kiểm thử chức năng trò chuyện nhóm và phát sóng đa điểm:

```
py peer.py --name Carol --port 5003
```

- Bước 5: Khởi động Giao diện đồ họa (Tùy chọn) Để trải nghiệm tương tác thông qua giao diện người dùng, thực thi lệnh sau trên một cửa sổ mới:

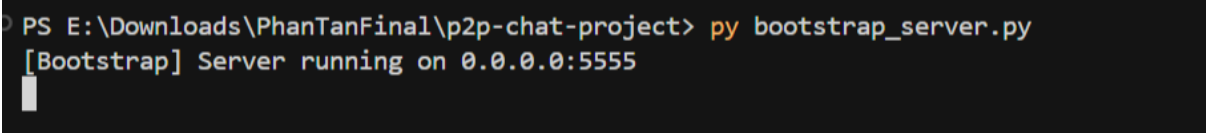
```
py peer_gui.py
```

Tại màn hình đăng nhập của ứng dụng, tiến hành thiết lập các thông số định tuyến ban đầu:

- Tên hiển thị: (Ví dụ: *Dave*)
- Cổng lắng nghe: (Ví dụ: *5004*)
- IP Bootstrap: 127.0.0.1
- Cổng Bootstrap: 5555

5.3. Demo

1. Khởi động máy chủ



```
PS E:\Downloads\PhanTanFinal\p2p-chat-project> py bootstrap_server.py
[Bootstrap] Server running on 0.0.0.0:5555
```

2. Khởi động nút mạng thứ nhất

VD:

- Tên: Alice
- Port: 5001
- IP Bootstrap: 127.0.0.1
- Cổng Bootstrap: 5555

Khởi động nút mạng thứ hai

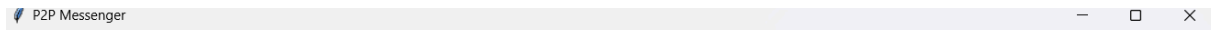
VD:

- Tên: Hai
- Port: 5002
- IP Bootstrap: 127.0.0.1
- Cổng Bootstrap: 5555

Khởi động nút mạng thứ ba

VD:

- Tên: Thai
- Port: 5003
- IP Bootstrap: 127.0.0.1
- Cổng Bootstrap: 5555



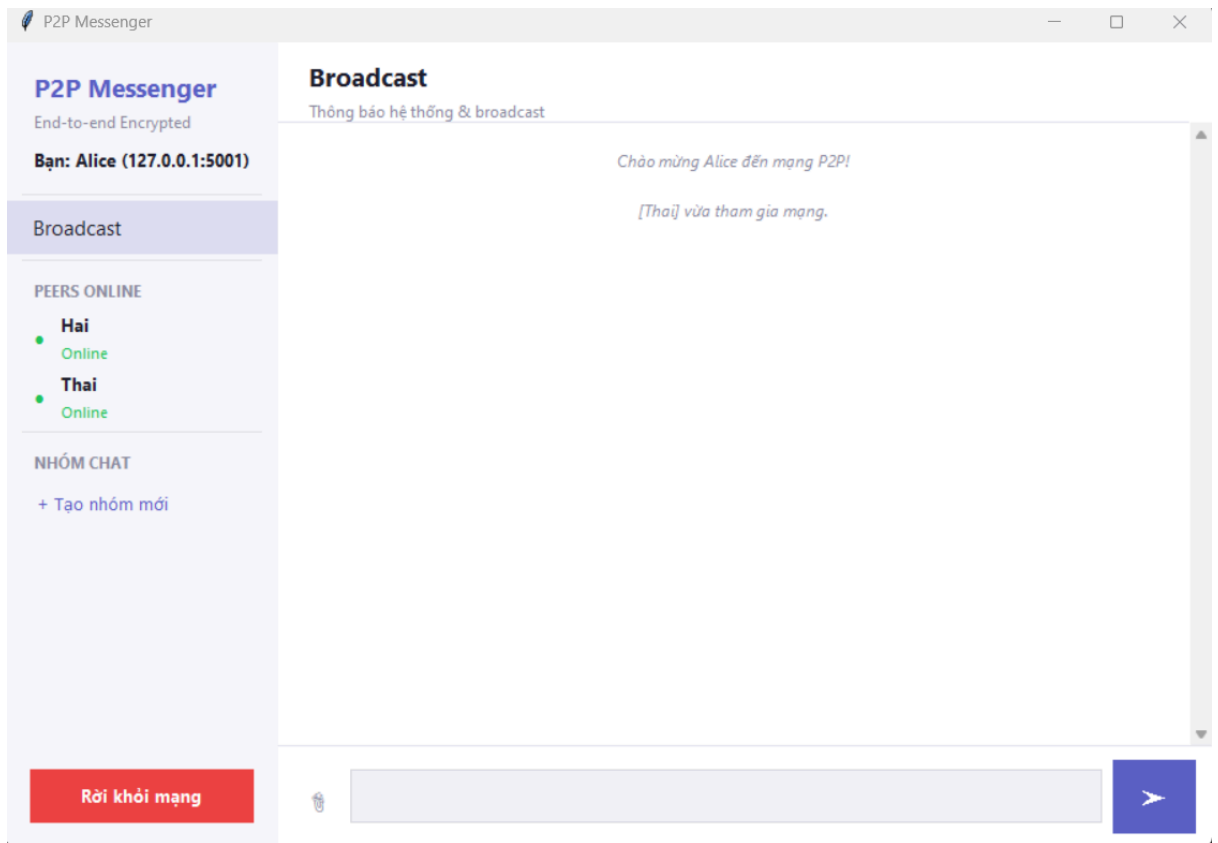
P2P Messenger

End-to-end Encrypted

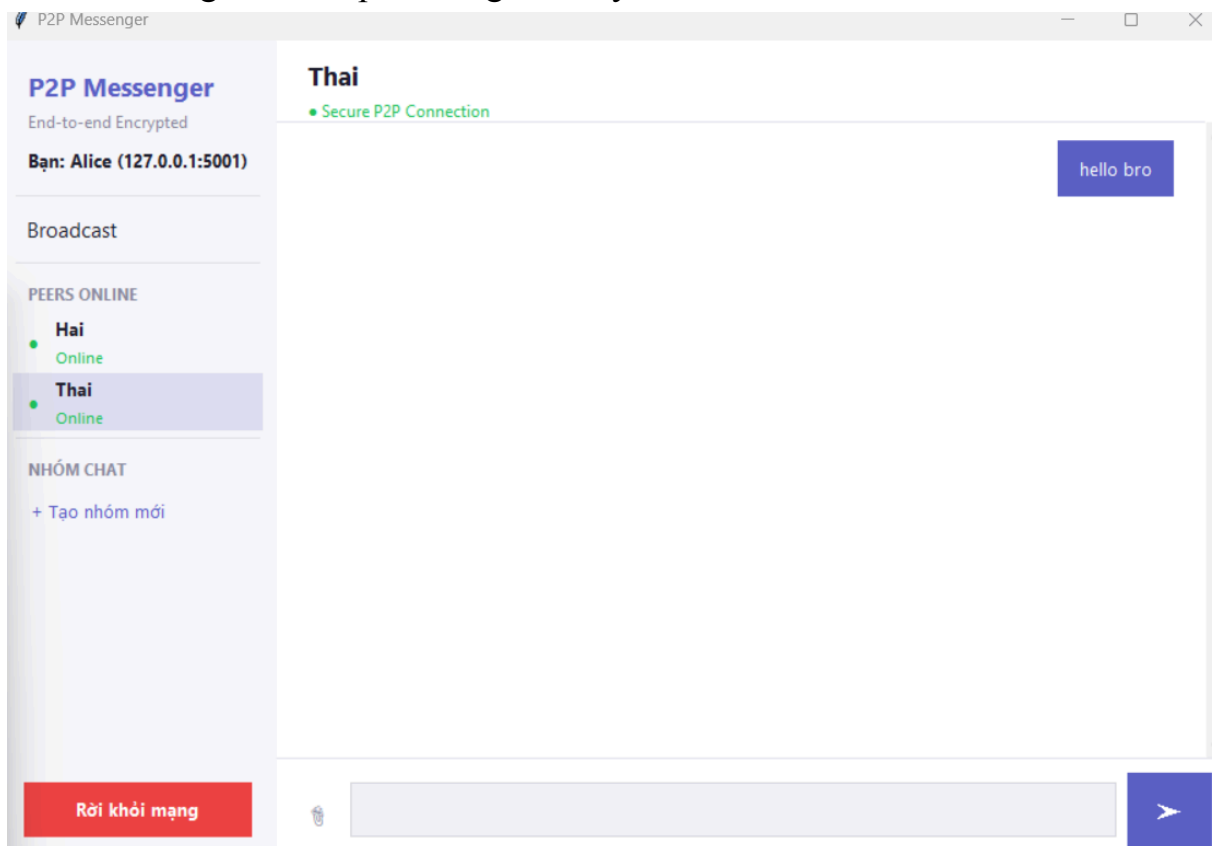
Tên hiển thị	Alice
Cổng lắng nghe	5001
IP Bootstrap	127.0.0.1
Cổng Bootstrap	5555

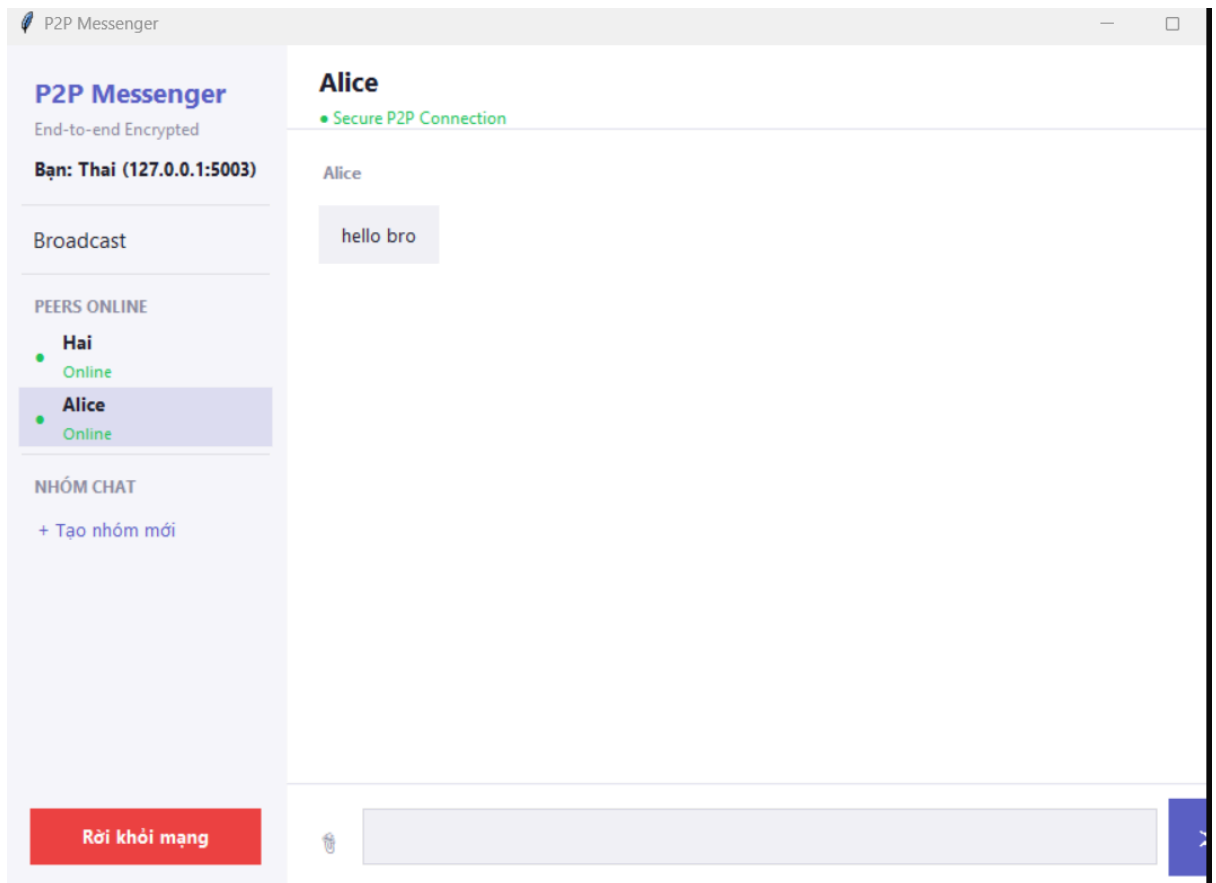
Kết nối vào mạng

3. Giao diện khi kết nối vào mạng

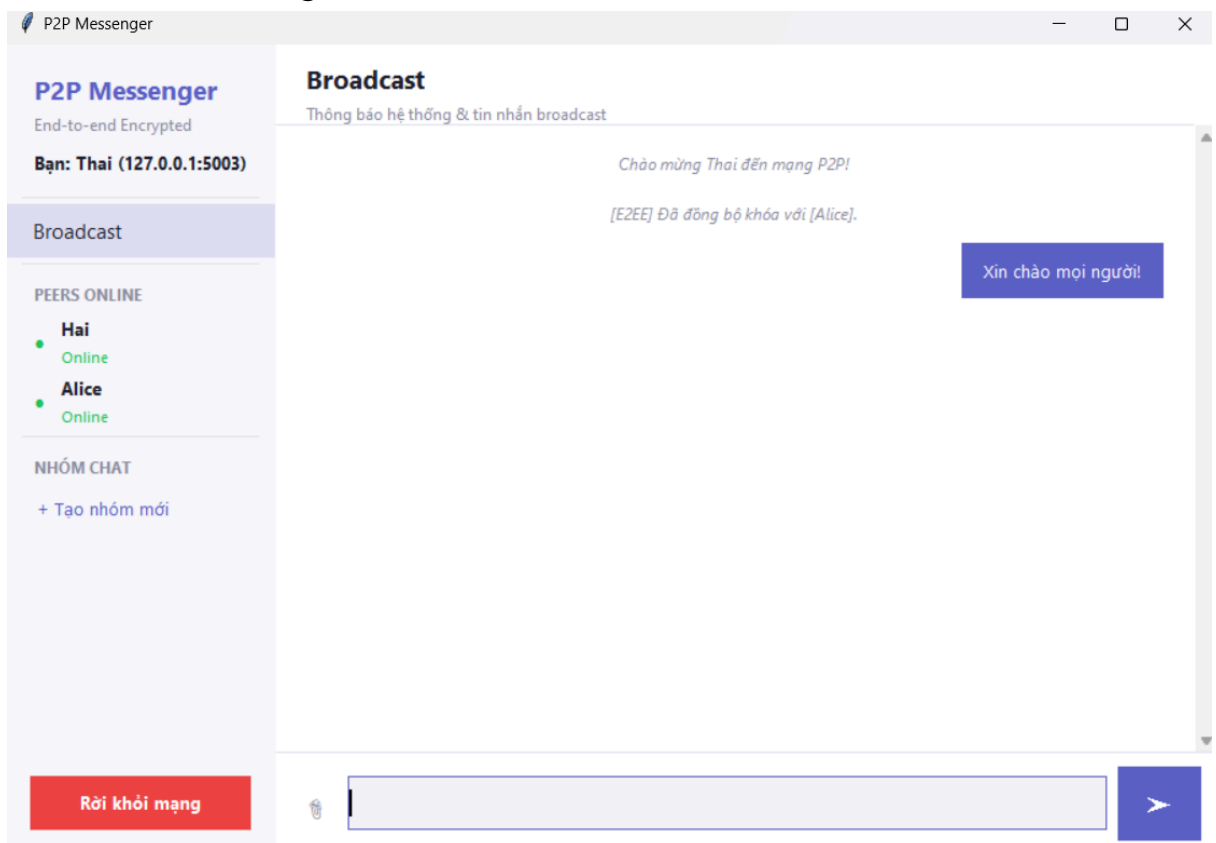


4. Nhắn tin riêng cho một peer đang trực tuyến

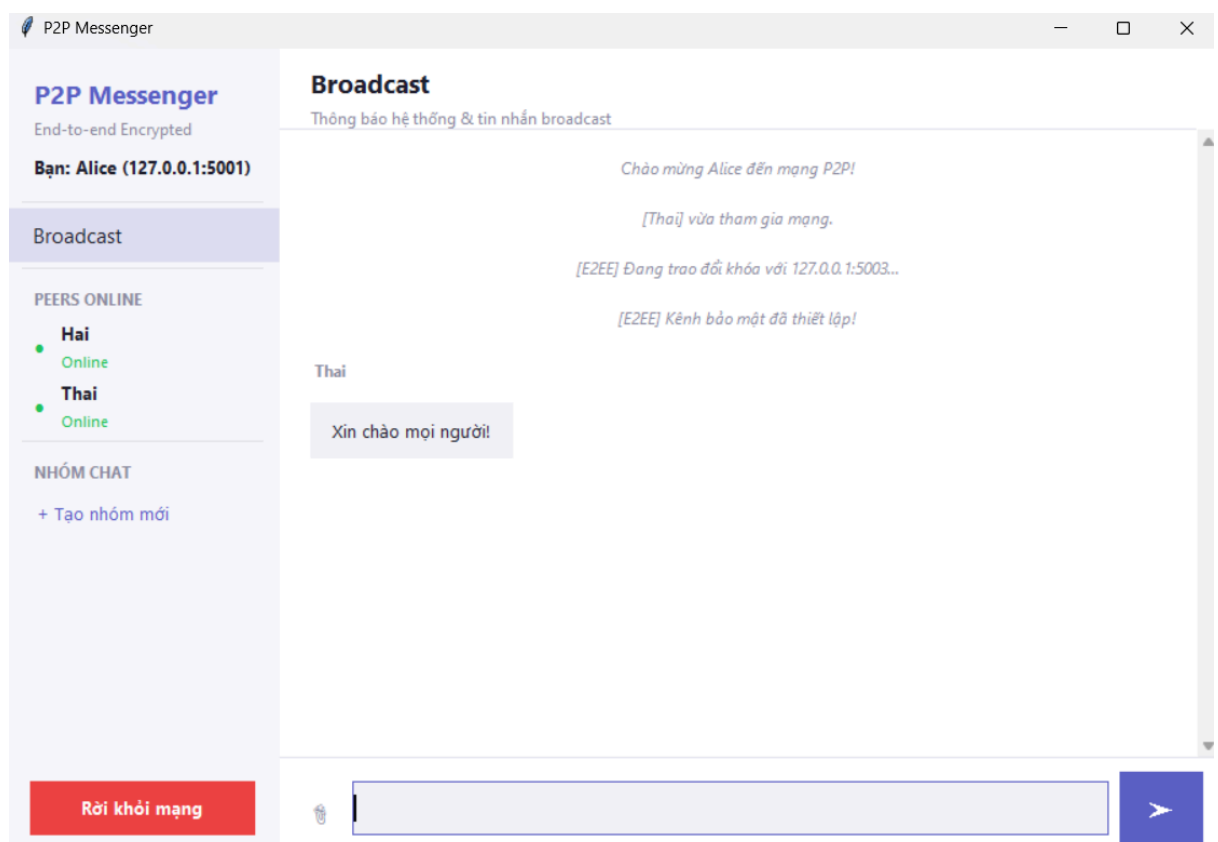
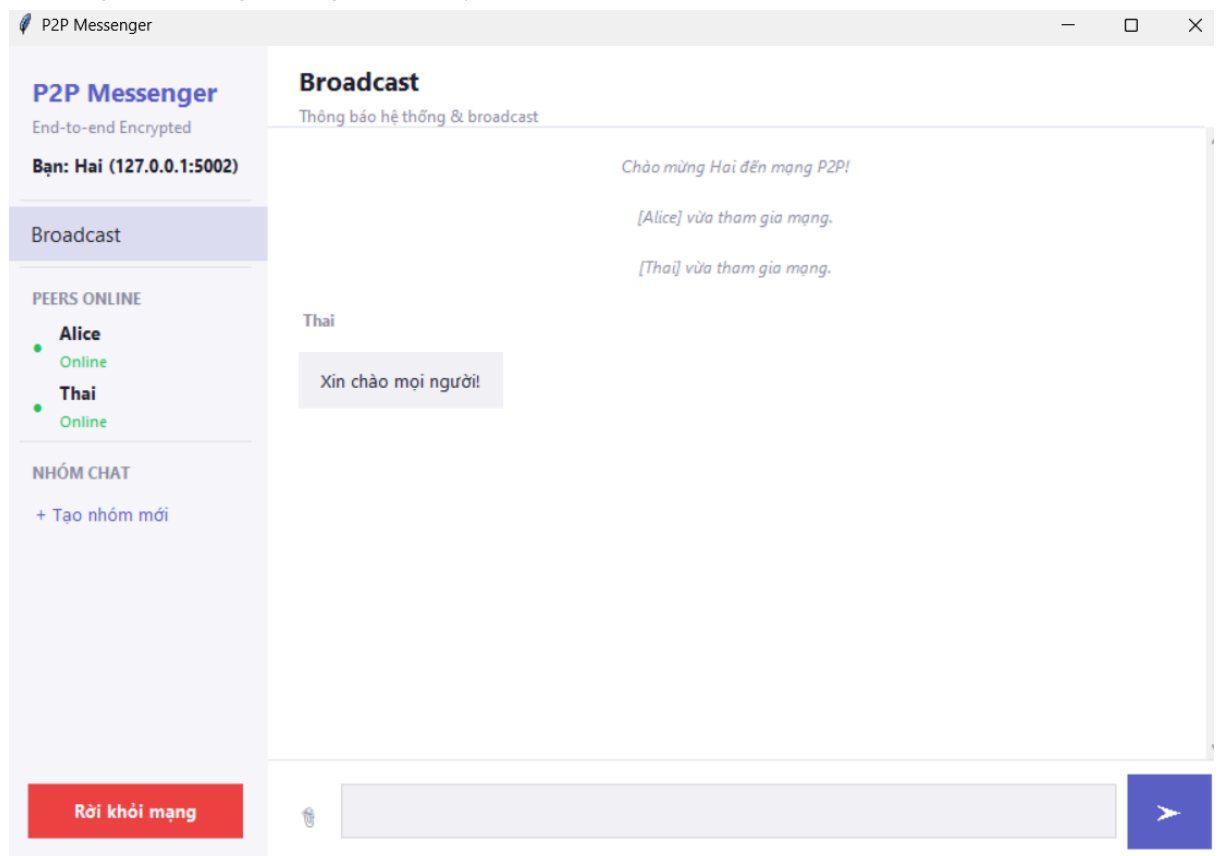




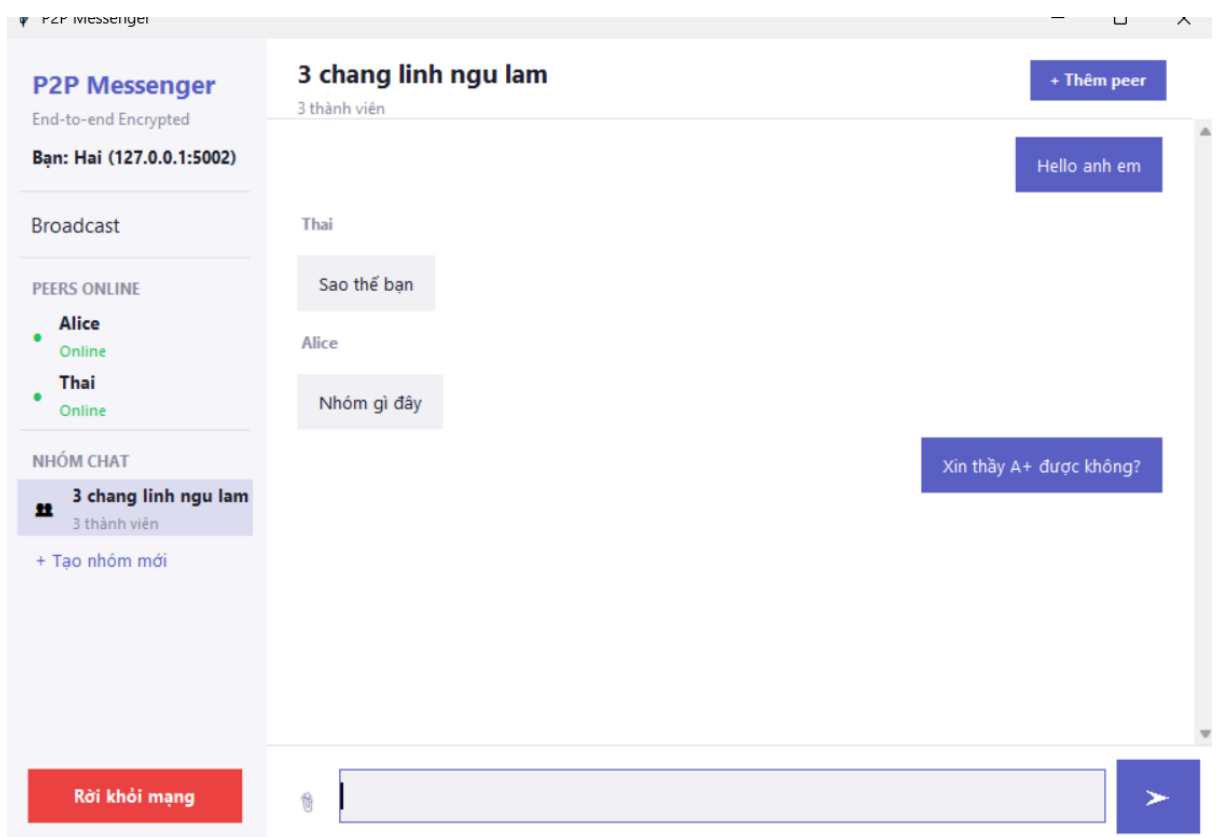
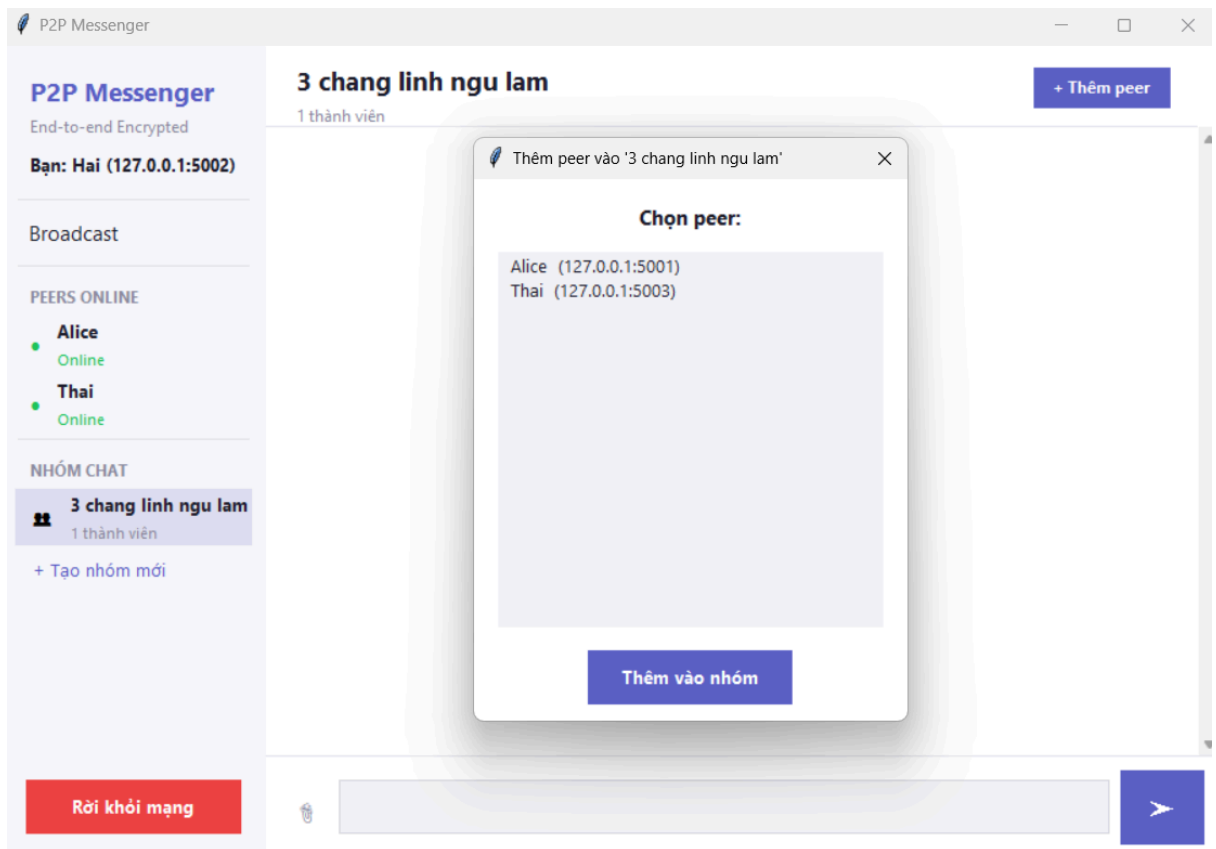
5. Broadcast toàn mạng



Mọi người trong mạng đều thấy

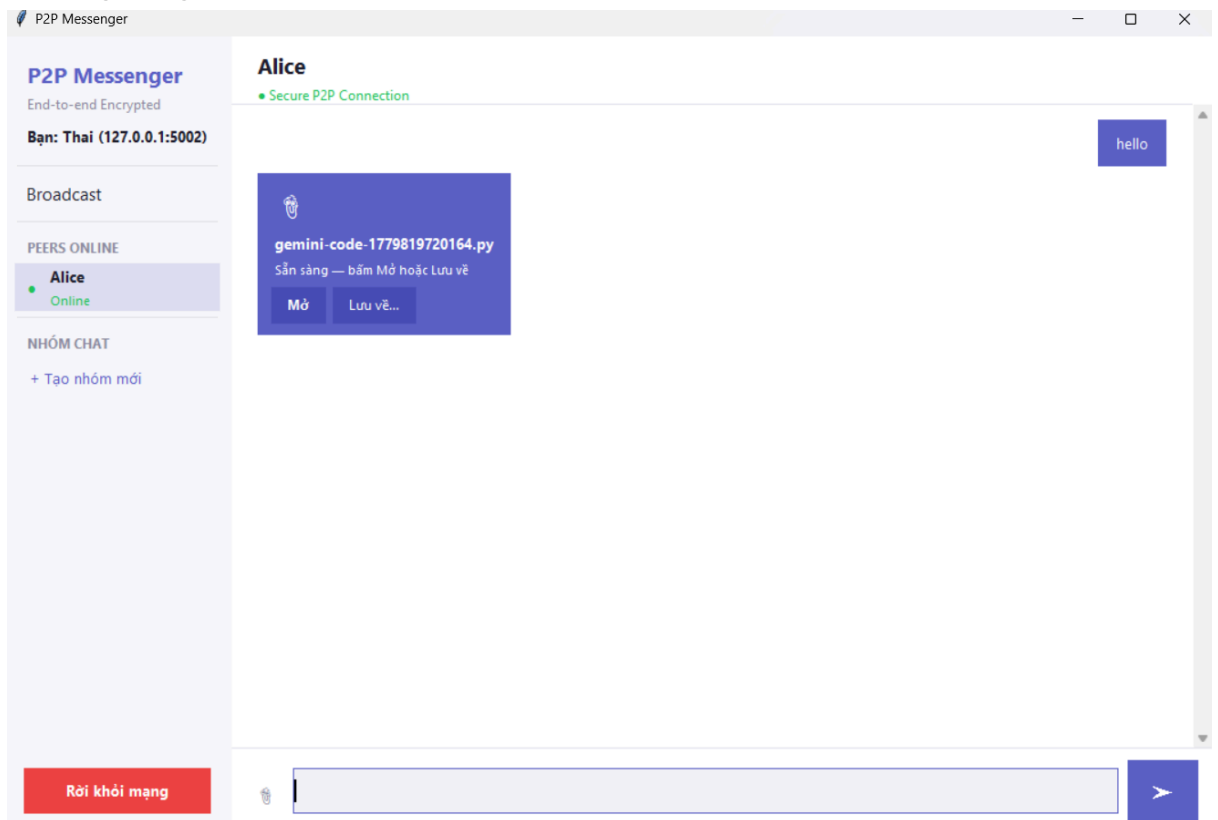


6. Tạo nhóm và sử dụng nhóm chat

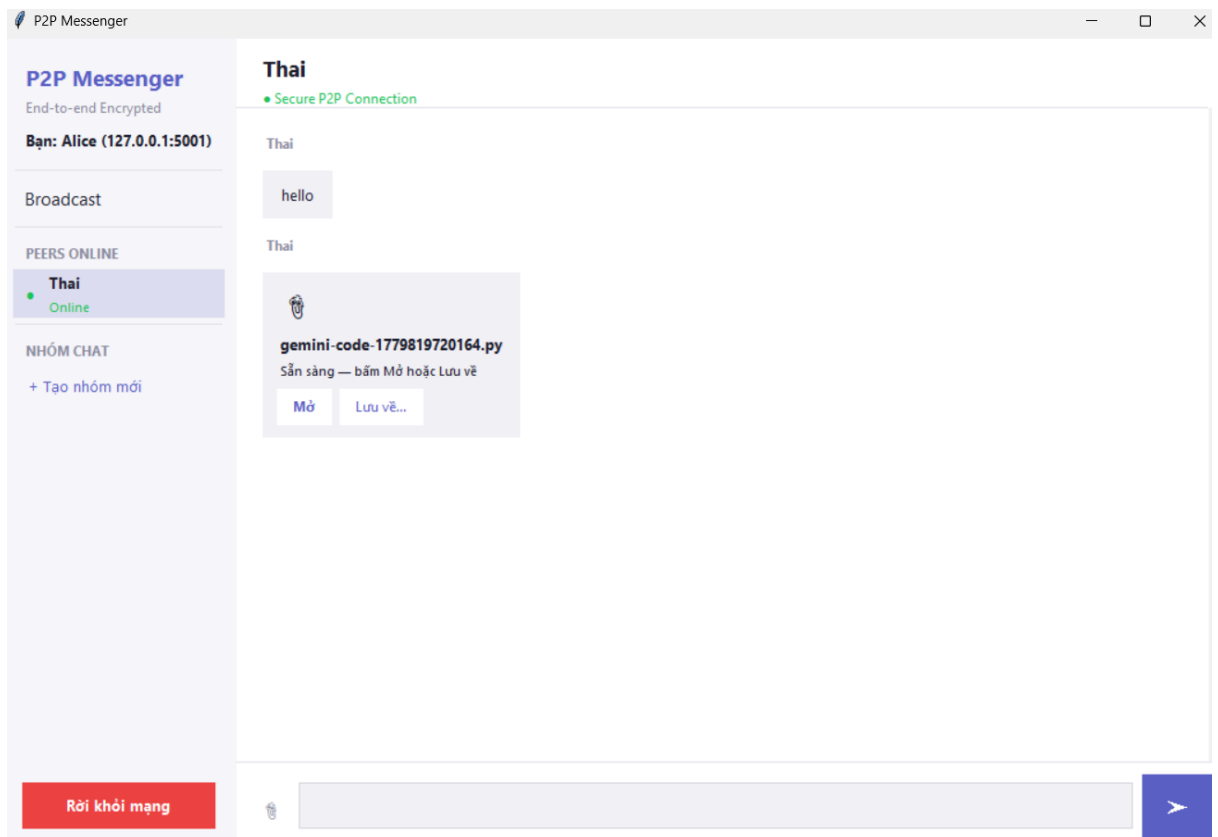


7. Gửi file

Phía người gửi:



Phía người nhận:



Chương 6: Đánh giá và hướng phát triển

6.1. Đánh giá kết quả đạt được

Về ưu điểm:

- Kiến trúc phân tán chuẩn mực: Dự án đã chứng minh thành công việc phân tách hoàn toàn Mặt phẳng điều khiển (Bootstrap) và Mặt phẳng dữ liệu (Peer-to-Peer). Hệ thống duy trì được luồng đàm thoại trực tiếp ngay cả khi cấu trúc mạng có sự biến động lớn.
- Bảo mật độc lập: Triển khai thành công cơ chế mã hóa đầu cuối (E2EE) qua thuật toán Diffie-Hellman. Máy chủ trung tâm đóng vai trò hộp thư mù (Store-and-forward) xuất sắc mà không xâm phạm quyền riêng tư của dữ liệu.
- Độ tin cậy tầng ứng dụng: Giải quyết tốt bài toán rút mạng chậm chờn thông qua cơ chế ACK kết hợp độ trễ tăng lũy thừa, đảm bảo tính toàn vẹn của thông điệp.

Về hạn chế:

- Nút thắt định dạng dữ liệu: Việc sử dụng chuỗi JSON với chuẩn Base64 gây lãng phí tài nguyên băng thông và buộc hệ thống phải áp đặt giới hạn dung lượng truyền tệp tin (tối đa 2MB).
- Rào cản hạ tầng mạng: Hệ thống hiện tại chưa tích hợp các cơ chế vượt tường lửa hoặc dịch địa chỉ mạng, do đó chỉ khả thi khi triển khai trên môi trường mạng cục bộ hoặc máy ảo cục bộ.
- Phụ thuộc điểm lỗi đơn bán phần: Dù luồng dữ liệu đã được phân tán, nhưng khâu định tuyến ban đầu và lưu trữ ngoại tuyến vẫn phụ thuộc vào một máy chủ Bootstrap duy nhất.

6.2. Hướng phát triển và mở rộng

Để hoàn thiện và nâng cấp hệ thống đáp ứng các tiêu chuẩn công nghiệp nghiệp, dự án đề xuất các hướng phát triển tiếp theo bao gồm:

- Tối ưu hóa giao thức: Thay thế định dạng JSON bằng các giao thức tuần tự hóa nhị phân (như Protocol Buffers hoặc MessagePack) nhằm giảm thiểu kích thước gói tin và hỗ trợ truyền tải các tệp tin đa phương tiện dung lượng lớn.

- Triển khai mạng diện rộng: Nghiên cứu và áp dụng các kỹ thuật NAT Hole Punching (thông qua máy chủ STUN/TURN) để các nút mạng có thể trực tiếp kết nối với nhau qua môi trường Internet toàn cầu.
- Phân tán hóa toàn diện: Tích hợp công nghệ Bảng băm phân tán thay thế hoàn toàn máy chủ Bootstrap, biến hệ thống thành một mạng lưới ngang hàng thuần túy không có điểm lỗi đơn.
- Lưu trữ bền vững: Tích hợp các hệ quản trị cơ sở dữ liệu nhúng cục bộ (như SQLite) trên từng thiết bị người dùng nhằm bảo toàn lịch sử hội thoại và trạng thái nhóm thay vì chỉ lưu trữ tạm thời trên bộ nhớ RAM.